

Every question you have to write, answered in full

---

## What is in here

23 questions and 51 worked answers, from 9 papers and practice sets: the examiner's own past papers, the mock papers, and every written question in the Learn by doing section. Each one is quoted as printed, broken down, then answered in full.

The objective half lives in the [question book](#).

## Learn by Doing: Topic One

Topic One

## Part B: the written questions, in the shape this paper sets them

---

### Question One: What Python is, and a program that will not run

---

#### BREAK IT DOWN FIRST

before you write a word

There are four separate faults in five lines, and two of them are the same kind of fault. Find all four before you read on, and say which one Python reports first.

#### ANSWER

**Fault one** The if line has no colon, so the block never opens. Python reports this first, as a `SyntaxError`, and stops: nothing else runs.

**Fault two** The two print lines are not indented under the if, so even with a colon the block would be empty.

**Fault three** Two values sit side by side inside print with nothing between them, neither a comma nor a plus.

**Fault four** age came from input, so it is a `STRING`. Comparing it with 17 and adding 1 to it both fail, and the fix is `int()`.

**Marks** Part (c) is three marks for four faults, so name them in a list, one line each, and do not explain at length.

#### PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

(a) What is Python, with four features. (b) How a Python program is executed, against a compiled language. (c) The errors in the student's program. (d) A corrected version.

**(a) What Python is.** Python is a high level, general purpose programming language that is **interpreted** rather than compiled, and it is designed to be readable.

Four features:

1. **Interpreted:** the source is executed line by line by the Python interpreter, with no separate compile step.
2. **Dynamically typed:** a variable takes the type of whatever is assigned to it, so no type is declared.
3. **Readable, indentation defined syntax:** blocks are marked by indentation rather than by braces, so the layout is the structure.
4. **Cross platform, with a large standard library:** the same program runs on Windows, Linux or macOS, and modules such as math, random and sqlite3 ship with it.

Others that earn the mark: free and open source, supports several paradigms, and can be extended with modules.

**(b) How it is executed.** Python runs a program **top to bottom, one statement at a time**. The interpreter reads a statement, executes it, then moves to the next, so a line that has not been reached has not run, and a name must be defined before the line that uses it.

A compiled language such as C is translated, whole, into machine code by a compiler before it is run, and the program you run is the translated file. Two consequences the examiner looks for: a compiled language catches many errors before the program ever runs, while Python reports most of them only when the line is reached; and Python must be installed to run the source, while a compiled binary runs on its own.

One exception worth a mark: a syntax error is found when the file is first parsed, before any line runs at all, which is exactly what happens in part (c).

**(c) The errors.**

| Line               | Error                                                                                                                                                                                                                   |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3                  | <b>No colon</b> at the end of the if statement. A statement that opens a block must end in a colon, so this is a SyntaxError and the program does not start.                                                            |
| 4 and 5            | <b>No indentation.</b> The body of the if must be indented under it. As written, the if has an empty body and the prints are not part of it.                                                                            |
| 4                  | <b>Two values with nothing between them</b> inside print: "Hello " name. It needs a comma, or a plus to join the strings, or an f-string.                                                                               |
| 2, then<br>3 and 5 | <b>age is a string</b> , because input always returns a string. So age > 17 compares a string with an integer, which raises a TypeError, and age + 1 tries to add an integer to a string, which raises a TypeError too. |

**(d) The corrected program.**

fixed.py

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
```

```
if age > 17:
    print("Hello", name)
    print("Next year you will be", age + 1)
```

```
Enter your name: Ada
Enter your age: 19
Hello Ada
Next year you will be 20
```

The conversion is done once, on the way in, which is the habit to keep: read with input, convert immediately with int or float, and the rest of the program can then do arithmetic without thinking about it.

## Question Two: Reading input, converting it, and printing a result

### BREAK IT DOWN FIRST

before you write a word

Which two lines of this program carry all five marks in part (b), and what does part (c) want that most students forget to supply?

#### ANSWER

**The two lines** The input lines, wrapped in int(). If the conversion is missing, the multiplication silently repeats a string instead of costing anything, and the program is wrong without crashing.

**Part (c)** The actual printed lines, exactly as they would appear, prompts included. Write the output as a block, not as a sentence describing it.

**Worth a mark cheaply** Formatting money to two decimal places with an f-string, and labelling each figure.

### PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

(a) Keyword, identifier and literal, with examples. (b) A program that reads black and white and colour page counts and prints each cost and the total. (c) Its output for 10 and 3.

#### (a) The three terms.

| Term       | What it is                                                                                                                                                              | Example                   |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|
| Keyword    | A word reserved by the language itself, which cannot be used as a name                                                                                                  | if, for, def, while, True |
| Identifier | A name the programmer gives to a variable, function, class or module. It may contain letters, digits and underscores, may not begin with a digit, and is case sensitive | bw_pages, total_cost      |

**Literal** A fixed value written directly in the code

20, 50.0,  
"colour", True

### (b) The program.

printing.py

```
BW_RATE = 20
COLOUR_RATE = 50

bw = int(input("Black and white pages: "))
colour = int(input("Colour pages: "))

bw_cost = bw * BW_RATE
colour_cost = colour * COLOUR_RATE
total = bw_cost + colour_cost

print(f"Black and white: {bw} pages at {BW_RATE} naira = {bw_cost:.2f}")
print(f"Colour: {colour} pages at {COLOUR_RATE} naira = {colour_cost:.2f}")
print(f"Total: {total:.2f}")
```

```
Black and white pages: 10
Colour pages: 3
Black and white: 10 pages at 20 naira = 200.00
Colour: 3 pages at 50 naira = 150.00
Total: 350.00
```

**(c) The output** is the block above, produced by running it with 10 and 3. Note three things the examiner can see at a glance: the rates are named constants rather than numbers buried in the arithmetic, both inputs are converted with `int` on the way in, and the money is formatted to two decimal places with `:.2f`.

## Learn by Doing: Topic Two

Topic Two

### Part B: the written questions, in the shape this paper sets them

#### Question One: The operators, conversion, and a listing to trace

##### BREAK IT DOWN FIRST

before you write a word

Two of these parts are the same skill in different clothes. Which two, and what single arithmetic fact answers both the second printed line and the whole of part (d)?

##### ANSWER

**The same skill** Part (c) and part (d). Both turn on what floor division and modulus actually give you.



**The fact** Floor division gives how many WHOLE times one number goes into another, and modulus gives what is LEFT OVER. Every conversion of a total into units uses the pair together.

**Watch in (c)** The first line prints 3.4, with a decimal point, because a single slash always returns a float, even when it divides exactly.

**Watch in (e)** and and or hand back one of their OPERANDS rather than True or False. Here both operands are already booleans, so booleans print, but the examiner may ask about zero or an empty string.

## PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

(a) Division, floor division, modulus and exponent. (b) Type conversion, and int of the string 3.5. (c) Trace the listing. (d) A seconds to hours, minutes and seconds program. (e) How and, or and not work.

### (a) The four arithmetic operators.

| Operator          | What it does                                                                   | Example                          |
|-------------------|--------------------------------------------------------------------------------|----------------------------------|
| / division        | True division. <b>Always returns a float</b> , even when the division is exact | 17 / 5 is 3.4, and 10 / 2 is 5.0 |
| // floor division | Divides and throws the fraction away, giving how many whole times it goes in   | 17 // 5 is 3                     |
| % modulus         | The remainder after that division                                              | 17 % 5 is 2                      |
| ** exponent       | Raises to a power. This is the power operator in Python, not the caret         | 17 ** 2 is 289                   |

```
print(17 / 5, 17 // 5, 17 % 5, 17 ** 2)
print(10 / 2, type(10 / 2))
```

```
3.4 3 2 289
5.0 <class 'float'>
```

**(b) Type conversion.** Type conversion, or casting, is changing a value from one type to another with a built in function: `int()`, `float()`, `str()`, `bool()`. It matters most on input, because **input always returns a string**.

Converting the string 3.5 with `int()` **raises a ValueError**. `int` will parse a string of digits, but it will not parse a decimal point: it neither rounds nor truncates a string. Convert to float first, then to `int` if a whole number is wanted:

```
print(float("3.5"))
print(int(float("3.5")))
try:
    print(int("3.5"))
```

```
except ValueError as e:
    print("ValueError:", e)
```

3.5

3

ValueError: invalid literal for int() with base 10: '3.5'

Note that `int(3.5)`, on the NUMBER, is legal and gives 3: it truncates towards zero rather than rounding.

### (c) What the listing prints.

```
a = 17
b = 5

print(a / b)
print(a // b)
print(a % b)
print(a ** 2)
print(a > b and a % 2 == 0)
print(a > b or a % 2 == 0)
```

3.4

3

2

289

False

True

Line by line: 3.4, because a single slash gives a float; 3, because floor division drops the fraction; 2, the remainder; 289, the power; then 17 is greater than 5, which is True, but 17 leaves a remainder of 1 when divided by 2, so the second test is False. **and** needs both, so False. **or** needs one, so True.

### (d) Seconds to hours, minutes and seconds.

seconds.py

```
total = int(input("Seconds: "))

hours = total // 3600
minutes = (total % 3600) // 60
seconds = total % 60

print(f"{total} seconds is {hours} h {minutes} m {seconds} s")
```

Seconds: 7325

7325 seconds is 2 h 2 m 5 s

Learn the shape, not the numbers: **floor divide to take the whole units out, modulus to keep what is left**, then repeat on the remainder. The same three lines turn kobo into naira or pages into reams.

### (e) and, or and not.

| Operator | How it works                                                                                                                                                                                       |
|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| and      | True only if BOTH sides are true. It <b>short circuits</b> : if the left side is false, the right side is never evaluated. It returns the first falsy operand, or the last one when all are truthy |
| or       | True if EITHER side is true. It short circuits too: if the left side is true, the right is never evaluated. It returns the first truthy operand, or the last when all are falsy                    |
| not      | Reverses a truth value, so not True is False                                                                                                                                                       |

```
age = 20
print(age > 17 and age < 65)
print(age < 17 or age > 65)
print(not age > 17)
print(0 or "default")
print("given" and "second")
```

True  
False  
False  
default  
second

The last two lines are the detail that separates a full answer: zero or a default gives back the default itself, not True, because or hands back an operand. That is how a fallback value is written in one line.

## Question Two: Strings: what they are, slicing them, and a listing to fix

### BREAK IT DOWN FIRST

before you write a word

Part (b) asks for five slices. What two rules answer all five, and what does a negative index mean?

#### ANSWER

**Rule one** A slice is start, stop, step. The **start is included and the stop is excluded**, so positions 0 to 6 give six characters, 0 to 5.

**Rule two** Leave a side out and it runs to the end of the string on that side: nothing before the colon means from the beginning, nothing after it means to the end.

**Negative index** Counts from the right, starting at minus one for the last character. A step of minus one walks the whole string backwards, which is how a string is reversed.

**Part (e)** Three faults, not two: assigning to a character, calling a method without its brackets, and two values inside print with nothing between them.

(a) What a string is, and immutability. (b) Five slices of PYTHON PROGRAMMING. (c) Four string methods with examples. (d) A program printing a word reversed with its vowel count. (e) The errors in the listing.

**(a) What a string is.** A string is an **ordered, immutable sequence of characters**, written in single or double quotes. Being a sequence, it can be indexed, sliced and looped over, and `len()` gives its length.

**Immutable** means that once a string exists it cannot be changed in place. Anything that looks like a change actually builds and returns a NEW string and leaves the original alone. So assigning to one character is an error:

```
name = "ada lovelace"
try:
    name[0] = "A"
except TypeError as e:
    print("TypeError:", e)

upper = name.upper()
print(upper)
print(name)
```

```
TypeError: 'str' object does not support item assignment
ADA LOVELACE
ada lovelace
```

The last two lines prove it: upper returned a new string, and name is exactly as it was. To keep a change you must assign it back, as `name = name.upper()`.

**(b) The five slices.**

```
title = "PYTHON PROGRAMMING"
print(repr(title[0:6]))
print(repr(title[7:]))
print(repr(title[-11:]))
print(repr(title[:2]))
print(repr(title[::-1]))
```

```
'PYTHON'
'PROGRAMMING'
'PROGRAMMING'
'PTO RGAMN'
'GNIMMARGORP NOHTYP'
```

In words: the first six characters, because the stop is excluded; everything from position seven to the end; the last eleven characters, counted from the right; every second character; and the whole string backwards.

### (c) Four string methods.

| Method                         | What it does                                                                                                                     | Example                                                  |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| <code>upper()</code>           | Returns a copy with every letter in upper case, and <code>lower()</code> is its mirror                                           | ada becomes ADA                                          |
| <code>strip()</code>           | Returns a copy with leading and trailing whitespace removed, which is what you do to anything a user typed                       | two spaces around ada become nothing                     |
| <code>replace(old, new)</code> | Returns a copy with every occurrence of one substring replaced by another                                                        | 2025 with 5 replaced by 6 becomes 2026                   |
| <code>split(sep)</code>        | Breaks the string into a LIST of pieces at each separator, or at whitespace if none is given. <code>join</code> does the reverse | a,b,c split on the comma becomes a list of three strings |

Others that earn the mark: `find()`, `count()`, `startswith()`, `title()`, `isdigit()`. Every one of them **returns a new value and leaves the original alone**, which is immutability again.

```
print("ada".upper(), "  ada  ".strip(), "2025".replace("5", "6"), "a,b,c".split(","))
```

```
ADA ada 2026 ['a', 'b', 'c']
```

### (d) The program.

word.py

```
word = input("Enter a word: ").strip()

reversed_word = word[::-1]

vowels = 0
for letter in word.lower():
    if letter in "aeiou":
        vowels += 1

print(f"Reversed: {reversed_word}")
print(f"Vowels: {vowels}")
```

```
Enter a word: Programming
Reversed: gnimmargorP
Vowels: 3
```

Three habits on show: strip what the user typed, reverse with a slice rather than a loop, and lower the word before testing so a capital A still counts. `letter in "aeiou"` is the short way to ask whether a character is one of a set.

### (e) The errors in the listing.

| Line | Error |
|------|-------|
|------|-------|

- 2 It assigns to one character of a string. Strings are immutable, so this raises a `TypeError`. Build a new string instead, joining the new first letter to the rest with a slice
- 3 The method has no brackets, so it prints the method object rather than calling it. It must be `name.upper()`
- 4 Two values inside print with nothing between them. It needs a comma

fixed.py

```
name = "ada lovelace"
name = "A" + name[1:]
print(name.upper())
print("Length:", len(name))
```

```
ADA LOVELACE
Length: 12
```

## Learn by Doing: Topic Three

Topic Three

### Part B: the written questions, in the shape this paper sets them

---

#### Question One: The if ladder, the two kinds of loop, and a program to write

---

##### BREAK IT DOWN FIRST

before you write a word

In part (d), what goes wrong if you test for the lowest grade first? And in part (e), what are the three lines every accumulator program has?

##### ANSWER

**Order** An elif ladder stops at the FIRST true test. Test below 45 first and every score under 45 is caught, but a 90 falls through every test to the end. Start from one end and move steadily to the other, and never leave a gap.

**The accumulator** Three lines: set the total to zero BEFORE the loop, add to it INSIDE the loop, print it AFTER the loop. Put the initialisation inside the loop and it resets every pass.

**Validation** Part (d) says reject an impossible score, so one test comes before the ladder, not inside it.

**In (c)** continue jumps to the next pass of the loop, skipping the rest of the body. Since n is increased before it, the loop still ends.

##### PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

(a) The if, elif, else ladder and why order matters. (b) Count controlled against condition controlled loops. (c) Trace the listing and explain continue. (d) A grade program with validation. (e) A sum of even numbers program.

**(a) The if, elif and else ladder.** if tests a condition and runs its indented block when the condition is true. elif offers another test, tried only when every test above it was false. else has no test and runs when all of them were false. Each of those lines ends in a colon, and the block belonging to it is the indented lines under it.

**Why the order matters:** the ladder stops at the **first true test** and every branch below it is skipped. So the tests must be arranged so that the first one that can be true is the one you want. Written from the top down, a grade ladder must go from the highest boundary to the lowest, or a score of 90 would match a test for 45 and above and be graded D.

```
score = 90

if score >= 45:
    print("wrong ladder says D")
elif score >= 70:
    print("A, never reached")

if score >= 70:
    print("right ladder says A")
elif score >= 45:
    print("D, not reached")
```

wrong ladder says D  
right ladder says A

### (b) The two kinds of loop.

|              | Count controlled                                                  | Condition controlled                                                          |
|--------------|-------------------------------------------------------------------|-------------------------------------------------------------------------------|
| What it is   | Repeats a KNOWN number of times, once for each item in a sequence | Repeats WHILE a condition stays true, however many times that turns out to be |
| Written with | for, usually over range() or a list                               | while                                                                         |
| The risk     | Off by one: range stops one before its second number              | An infinite loop, if nothing inside changes the condition                     |

```
for i in range(1, 6):
    print("count controlled", i)

n = 1
while n * n < 20:
    print("condition controlled", n, n * n)
    n = n + 1
```

```
count controlled 1
count controlled 2
count controlled 3
count controlled 4
count controlled 5
condition controlled 1 1
condition controlled 2 4
condition controlled 3 9
condition controlled 4 16
```

The for loop runs exactly five times, decided before it starts. The while loop runs until the square reaches 20, which you would have to work out, and **the line that changes n is what makes it end.**

**(c) The trace.**

```
total = 0
n = 1

while n <= 10:
    if n % 3 == 0:
        n = n + 1
        continue
    total = total + n
    n = n + 1

print(total, n)
```

37 11

The loop adds every number from 1 to 10 except the multiples of three, which are 3, 6 and 9. The numbers added are 1, 2, 4, 5, 7, 8 and 10, giving 37, and the loop leaves n at 11, which is the value that failed the test.

**What continue did:** it abandoned the rest of that pass and jumped straight to the next test of the condition, so the line that adds to the total was skipped for 3, 6 and 9. Note the deliberate detail: n is increased BEFORE the continue. Had it not been, n would have stayed at 3 forever and the program would never end.

**(d) The grade program.**

grade.py

```
score = int(input("Score: "))

if score < 0 or score > 100:
    print("Invalid score. It must be between 0 and 100.")
else:
    if score >= 70:
        grade = "A"
    elif score >= 60:
        grade = "B"
```



```

elif score >= 50:
    grade = "C"
elif score >= 45:
    grade = "D"
else:
    grade = "F"
print(f"Score {score} is grade {grade}")

```

Score: 64

Score 64 is grade B

Validation first, in its own test, then the ladder from the highest boundary down. Because each elif is only reached when the ones above failed, no upper bound is needed in the tests: by the time score >= 60 is reached, the score is already known to be under 70.

### (e) The sum of the even numbers.

evens.py

```

n = int(input("n: "))

total = 0
count = 0

for number in range(1, n + 1):
    if number % 2 == 0:
        total = total + number
        count = count + 1

print(f"The {count} even numbers up to {n} add up to {total}")

```

n: 10

The 5 even numbers up to 10 add up to 30

This is the accumulator and the counter in one program: both are set to zero before the loop, both are updated inside it, and both are printed after it. Note range(1, n + 1): range stops one before its second number, so the plus one is what makes n itself count.

## Question Two: Lists, tuples, sets and dictionaries, and marks to analyse

### BREAK IT DOWN FIRST

before you write a word

The first line of output in part (c) catches most people. What does copy = scores actually copy, and how would you make a real copy?

#### ANSWER

**What it copies** Nothing. It makes a second NAME for the same list, so appending through one name shows up through the other. That is aliasing.

**A real copy** `copy = scores[:]` or `copy = list(scores)`, either of which builds a new list.

**Why it only bites here** A list is **MUTABLE**. Aliasing a tuple or a string is harmless, because neither can be changed in place.

**The set in the last line** The sentence has six words but only five different ones, because the repeats collapse. That is what a set is for.

## PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

(a) List, tuple, set and dictionary compared. (b) Five list operations. (c) Trace the listing and explain its first line. (d) A program for average, highest and count above average. (e) A word frequency program.

### (a) The four containers.

|                   | Written                                       | Changeable                                   | What it is for                                                                                                                          |
|-------------------|-----------------------------------------------|----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <b>List</b>       | Square brackets, comma separated              | Yes, mutable                                 | An ordered collection you will add to, remove from or sort. The default choice                                                          |
| <b>Tuple</b>      | Round brackets                                | No, immutable                                | A fixed group of related values, such as a coordinate or a record that must not change. It can be a dictionary key, which a list cannot |
| <b>Set</b>        | Curly braces of values, or <code>set()</code> | Yes, but it holds no duplicates and no order | Membership and uniqueness: removing duplicates, or asking whether something is present                                                  |
| <b>Dictionary</b> | Curly braces of key colon value               | Yes                                          | Looking a value up BY A KEY rather than by position, such as a name to a mark                                                           |

```
marks = [45, 62, 70]
point = (3, 4)
unique = {"a", "b", "a"}
ages = {"ada": 36, "grace": 45}

print(marks, point, sorted(unique), ages["grace"])
print(type(marks), type(point), type(unique), type(ages))
```

```
[45, 62, 70] (3, 4) ['a', 'b'] 45
<class 'list'> <class 'tuple'> <class 'set'> <class 'dict'>
```

### (b) Five list operations.

| Operation                 | What it does                                                      | Example                                              |
|---------------------------|-------------------------------------------------------------------|------------------------------------------------------|
| <code>append(x)</code>    | Adds one item to the end                                          | a list of three becomes four                         |
| <code>insert(i, x)</code> | Puts an item at position <code>i</code> and shifts the rest along | <code>insert</code> at 0 makes it the new first item |

|                        |                                                                                                          |                                                           |
|------------------------|----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| <code>remove(x)</code> | Removes the FIRST item equal to x, and raises a <code>ValueError</code> if there is none                 | removes a value by what it is                             |
| <code>pop(i)</code>    | Removes the item at position i and RETURNS it. With no argument it takes the last one                    | removes a value by where it is                            |
| <code>sort()</code>    | Sorts the list IN PLACE and returns nothing. <code>sorted(list)</code> returns a new sorted list instead | ascending by default, descending with reverse set to true |

```
marks = [45, 62, 70]
marks.append(38)
marks.insert(0, 99)
marks.remove(62)
taken = marks.pop()
marks.sort()
print(marks, "popped", taken)
print(sorted(marks, reverse=True))
```

```
[45, 70, 99] popped 38
[99, 70, 45]
```

The pair worth naming: `remove` takes a VALUE, `pop` takes a POSITION and hands the item back. And `sort()` returns `None`, so `marks = marks.sort()` destroys the list, which is a favourite objective question.

### (c) The trace.

```
scores = [45, 62, 70, 38]
copy = scores
copy.append(90)

names = ("ada", "grace")
words = "the cat sat on the mat".split()

print(scores)
print(len(names), len(set(words)))
```

```
[45, 62, 70, 38, 90]
2 5
```

**The first line explained.** `copy = scores` did not copy the list. It gave the same list a second name, so appending through `copy` changed the one and only list, and printing `scores` shows the 90. This is **aliasing**, and it only matters for mutable containers. To take a real copy use `scores[:]` or `list(scores)`.

The second line: the tuple holds two names, and the sentence has six words of which five are different, since the repeated word collapses in the set.

### (d) The marks program.

marks.py

```

marks = []

for i in range(1, 6):
    mark = float(input(f"Mark {i}: "))
    marks.append(mark)

average = sum(marks) / len(marks)
highest = max(marks)

above = 0
for mark in marks:
    if mark > average:
        above += 1

print(f"Average: {average:.2f}")
print(f"Highest: {highest}")
print(f"Above average: {above}")

```

```

Mark 1: 45
Mark 2: 62
Mark 3: 70
Mark 4: 38
Mark 5: 55
Average: 54.00
Highest: 70.0
Above average: 3

```

Read into a list with append inside a count controlled loop, then let the built ins do the work: `sum()`, `len()` and `max()`. The count above the average needs its own loop, because the average is not known until every mark is in.

### **(e) The word count.**

wordcount.py

```

sentence = input("Sentence: ")
words = sentence.lower().split()

counts = {}
for word in words:
    counts[word] = counts.get(word, 0) + 1

for word, times in counts.items():
    print(f"{word}: {times}")

```

```

Sentence: the cat sat on the mat
the: 2
cat: 1
sat: 1
on: 1
mat: 1

```

The line that carries the marks is `counts.get(word, 0) + 1`: `get` returns the value for a key, or the default you give it when the key is absent, which is what saves you writing an `if` to test whether the word is already there. Indexing a missing key with square brackets would raise a **KeyError** instead. `items()` then walks the dictionary as key and value pairs.

## Learn by Doing: Topic Four

Topic Four

### Part B: the written questions, in the shape this paper sets them

#### Question One: Functions, scope, and a module you are given

##### BREAK IT DOWN FIRST

before you write a word

The last line of the listing prints two numbers. Why is the second one still zero, and what would you have to write to change it?

##### ANSWER

**Why zero** The assignment inside `tally` creates a NEW local variable that happens to share the name. A function cannot change a global by assigning to it; the local one dies when the function returns.

**To change it** Declare `global count` inside the function, or better, return the value and assign it at the call: `count = tally()`.

**The other catch** The default list in `add` is created ONCE, when the function is defined, so the second call appends to the same list and shows both items.

**Marks** Part (d) is four marks for two things: the function, and the loop that uses it. Write the function first and call it from a loop; do not write one long program with no function in it.

##### PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

(a) Functions, void against value returning, parameter against argument. (b) Default parameters and returning several values. (c) Trace the listing and explain its last line. (d) A prime testing function and a program that uses it. (e) Modules, two import forms, and a circle program.

**(a) Functions.** A function is a named block of code that performs one task and can be called as often as needed. It is defined with `def`, a name, brackets holding its parameters, and a colon, and its body is the indented block under it.

|              | Void function                                                | Value returning function                                    |
|--------------|--------------------------------------------------------------|-------------------------------------------------------------|
| What it does | Performs an action, such as printing, and hands nothing back | Computes a value and hands it back with <code>return</code> |

What the call is worth     **None**, which is what Python returns when there is no return statement

The returned value, which can be assigned or printed

Typical use     `show_menu()`

`area = circle_area(7)`

**Parameter against argument:** a **parameter** is the name in the function's definition; an **argument** is the actual value passed in the call. In `def cube(num)` the parameter is `num`; in the call `cube(4)` the argument is 4.

```
def show(name):
    print("Hello", name)

def cube(num):
    return num ** 3

result = show("Ada")
print("show returned:", result)
print("cube returned:", cube(4))
```

Hello Ada

show returned: None

cube returned: 64

**(b) Default parameters, and returning several values.** A parameter can be given a default in the definition, and the call may then leave it out. Defaults must come after the parameters that have none.

A function returns several values by returning them separated by commas, which makes a **tuple**, and the call can unpack it into several names at once.

```
def power(base, exponent=2):
    return base ** exponent

def stats(numbers):
    return min(numbers), max(numbers), sum(numbers) / len(numbers)

print(power(5), power(5, 3))

low, high, mean = stats([45, 62, 70])
print(low, high, round(mean, 2))
print(stats([45, 62, 70]))
```

25 125

45 70 59.0

(45, 70, 59.0)

**(c) The trace.**

```
count = 0

def add(item, basket=[]):
    basket.append(item)
```

```

    return basket

def tally():
    count = 1
    return count

print(add("bread"))
print(add("milk"))
print(tally(), count)

```

```

['bread']
['bread', 'milk']
1 0

```

**The last line explained.** The function returned 1, so the first number is 1. The global count is still 0, because the assignment inside the function created a **new local variable** with the same name, which was destroyed when the function returned. A function can READ a global, but assigning to that name inside it makes a local instead, unless the name is declared `global` first. The cleaner fix is the one the exam rewards: return the value and assign it at the call site, `count = tally()`.

**And the second line of output.** The list printed both items, although each call passed only one. A **default value is created once**, when the `def` is executed, so every call that leaves the parameter out shares the same list. It is why a mutable default is a bug: write `basket=None` and build a fresh list inside the function.

#### (d) The prime function and the program.

primes.py

```

def is_prime(number):
    if number < 2:
        return False
    for divisor in range(2, int(number ** 0.5) + 1):
        if number % divisor == 0:
            return False
    return True

limit = int(input("Up to: "))

for candidate in range(2, limit + 1):
    if is_prime(candidate):
        print(candidate, end=" ")

print()

```

```

Up to: 30
2 3 5 7 11 13 17 19 23 29

```

Three things earn the marks: the function does one job and returns True or False; return False leaves the moment a divisor is found, so nothing is tested after the answer is known; and the

loop only goes up to the square root, because a factor above it would already have been found below it. Testing every number up to  $n$  is still correct and still earns most of the marks.

**(e) Modules.** A module is a file of Python code, holding functions, constants and classes, that another program can bring in and use. The standard library ships with many: `math`, `random`, `datetime`, `os`, `sqlite3`.

Two ways to import:

1. `import math`, then use it as `math.pi`. The module name stays in front, so nothing clashes with your own names.
2. `from math import pi, sqrt`, then use `pi` directly. Shorter, but it can collide with a name of your own.

A third form, `import math as m`, renames it.

circle.py

```
import math
from math import sqrt

radius = float(input("Radius: "))

area = math.pi * radius ** 2
circumference = 2 * math.pi * radius

print(f"Area: {area:.2f}")
print(f"Circumference: {circumference:.2f}")
print(f"The radius from the area again: {sqrt(area / math.pi):.2f}")
```

```
Radius: 7
Area: 153.94
Circumference: 43.98
The radius from the area again: 7.00
```

## Question Two: Files, exceptions, and a program that must not crash

### BREAK IT DOWN FIRST

before you write a word

In part (c), which mode would silently destroy the file if the program were run twice, and in part (e), which two exception names must appear?

#### ANSWER

**The destructive mode** "w". Opening a file for writing EMPTIES it immediately, before a single line is written. To add to a file instead, open it with "a".

**The two exceptions** `ValueError`, raised by `int()` when the text is not a number, and `ZeroDivisionError`, raised by the division itself.



**The mark that is usually lost** Naming the exception. `except:` on its own catches everything, including your own typing mistakes, and the examiner wants the name.

**Reading back** A line read from a file ends in a newline, so strip it before printing or counting.

## PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

(a) Four file modes. (b) The `with` statement. (c) A program that writes three names to a file and reads them back numbered. (d) `try`, `except`, `else` and `finally`. (e) A division program that survives bad input and division by zero.

### (a) The file modes.

| Mode                | What it does                                                               | If the file does not exist |
|---------------------|----------------------------------------------------------------------------|----------------------------|
| "r" read            | Opens for reading only, positioned at the start. The default               | <b>FileNotFoundError</b>   |
| "w" write           | Opens for writing and <b>empties the file at once</b> , whatever was in it | Creates it                 |
| "a" append          | Opens for writing at the END, keeping what is already there                | Creates it                 |
| "r+" read and write | Opens for both, positioned at the start, without emptying it               | FileNotFoundError          |

Adding `b`, as in `"rb"`, opens the file in binary rather than text.

**(b) The `with` statement.** `with open("names.txt") as f:` opens the file and **closes it automatically** when the block ends, even if an error is raised inside the block. Without it you must call `f.close()` yourself, and any exception before that line leaves the file open, with data possibly still unwritten in the buffer. Fewer lines, and nothing to forget.

### (c) Writing and reading the names.

students.py

```
names = []
for i in range(1, 4):
    names.append(input(f"Name {i}: ").strip())

with open("students.txt", "w") as f:
    for name in names:
        f.write(name + "\n")

print("Saved. Reading it back:")

count = 0
with open("students.txt", "r") as f:
    for line in f:
        count += 1
```

```
print(f"{count}. {line.strip()}")

print(f"{count} names in the file")
```

```
Name 1: Ada
Name 2: Grace
Name 3: Chinwe
Saved. Reading it back:
1. Ada
2. Grace
3. Chinwe
3 names in the file
```

Four things the examiner is looking for: the file is opened with `with`; `"w"` to create it and `"r"` to read it; `write` does NOT add a line ending, so the newline is written explicitly; and each line read back is stripped, because it arrives with that newline still attached.

#### **(d) try, except, else and finally.**

| Part                 | What it is for                                                                                                                 |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <code>try</code>     | Holds the code that might raise an exception. Keep it as short as possible, so you know what failed                            |
| <code>except</code>  | Runs only if the named exception was raised, and stops the program from crashing. There may be several, one per exception name |
| <code>else</code>    | Runs only if NO exception was raised. It is where the work that depends on the <code>try</code> succeeding belongs             |
| <code>finally</code> | Runs either way, exception or not. It is for cleaning up, such as closing a file or a database connection                      |

An exception is an error raised while the program runs: `ValueError`, `ZeroDivisionError`, `FileNotFoundError`, `TypeError`, `IndexError`, `KeyError`. Catching one by name is the whole point; a bare `except`: hides every fault, including your own.

#### **(e) The division program.**

`divide.py`

```
try:
    a = float(input("First number: "))
    b = float(input("Second number: "))
    result = a / b
except ValueError:
    print("That was not a number.")
except ZeroDivisionError:
    print("You cannot divide by zero.")
else:
    print(f"{a} divided by {b} is {result:.2f}")
finally:
    print("Done.")
```

```
First number: 10
Second number: 4
10.0 divided by 4.0 is 2.50
Done.
```

Run again with a letter, and again with a zero, and the same program survives both:

both\_faults.py

```
for attempt in [("ten", "4"), ("10", "0")]:
    try:
        a = float(attempt[0])
        b = float(attempt[1])
        result = a / b
    except ValueError:
        print("That was not a number.")
    except ZeroDivisionError:
        print("You cannot divide by zero.")
    else:
        print(f"{a} divided by {b} is {result:.2f}")
    finally:
        print("Done.")
```

```
That was not a number.
Done.
You cannot divide by zero.
Done.
```

## Learn by Doing: Topic Five

Topic Five

### Part B: the written questions, in the shape this paper sets them

---

#### Question One: The SQLite skeleton, end to end

---

##### BREAK IT DOWN FIRST

before you write a word

Which single line, if you leave it out, makes the program appear to work and still lose everything?  
And what does a row come back as?

##### ANSWER

**The line** `conn.commit()`. Without it the inserts sit in an open transaction and are thrown away when the connection closes. The program prints no error at all.

**A row** comes back as a `TUPLE`, one value per selected column, so the rows are unpacked as a tuple, not as a dictionary.

**The other mark** IF NOT EXISTS on the create, so running the program twice does not raise an error.

**Write it in order** connect, cursor, execute, commit, select, fetchall, close. The order IS the answer to part (a).

## PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

(a) The steps for working with SQLite from Python. (b) A program that creates a students table, inserts three rows and prints them. (c) Placeholders, and why not to join values into SQL. (d) commit, and fetchone against fetchall.

### (a) The steps.

| Step                                                 | What it does                                                                          |
|------------------------------------------------------|---------------------------------------------------------------------------------------|
| <code>import sqlite3</code>                          | Brings in the module, which ships with Python                                         |
| <code>conn =<br/>sqlite3.connect("school.db")</code> | Opens the database file, <b>creating it if it does not exist</b>                      |
| <code>cur = conn.cursor()</code>                     | Creates a cursor, the object that carries statements to the database and results back |
| <code>cur.execute(sql)</code>                        | Runs one SQL statement: create, insert, update, delete or select                      |
| <code>conn.commit()</code>                           | Saves the changes permanently. Without it they are discarded                          |
| <code>cur.fetchall()</code>                          | After a select, hands back the rows, each as a tuple                                  |
| <code>conn.close()</code>                            | Closes the connection and releases the file                                           |

### (b) The program.

students\_db.py

```
import sqlite3

conn = sqlite3.connect("school.db")
cur = conn.cursor()

cur.execute('''CREATE TABLE IF NOT EXISTS students (
            matric TEXT PRIMARY KEY,
            name  TEXT NOT NULL,
            score REAL)''')

students = [("20/1234", "Ada Obi", 72.5),
            ("20/1235", "Grace Ede", 64.0),
            ("20/1236", "Chinwe Udo", 58.5)]

cur.executemany("INSERT INTO students VALUES (?, ?, ?)", students)
conn.commit()

cur.execute("SELECT matric, name, score FROM students ORDER BY score DESC")
```

```

rows = cur.fetchall()

print(f"{len(rows)} students")
for matric, name, score in rows:
    print(f"{matric} {name:<12} {score:.1f}")

conn.close()

```

```

3 students
20/1234  Ada Obi      72.5
20/1235  Grace Ede    64.0
20/1236  Chinwe Udo   58.5

```

Four details the examiner looks for: **IF NOT EXISTS**, so a second run does not fail; **question mark placeholders** rather than values built into the string; **commit** after the inserts; and **close** at the end. `executemany` runs the same statement once per row in a list, and `execute` would do it one at a time.

**(c) Placeholders.** A placeholder is a question mark written where a value belongs in the SQL, with the actual values passed as a second argument to `execute`, as a tuple:

```

import sqlite3

conn = sqlite3.connect("demo.db")
cur = conn.cursor()
cur.execute("CREATE TABLE students (matric TEXT, name TEXT, score REAL)")

wanted = 60
cur.execute("INSERT INTO students VALUES (?, ?, ?)", ("20/1237", "Ngozi Eze", 81.0))
conn.commit()

cur.execute("SELECT name, score FROM students WHERE score > ?", (wanted,))
print(cur.fetchall())
conn.close()

```

```
[('Ngozi Eze', 81.0)]
```

**Why never join values into the string.** Two reasons, and both earn marks. First, **SQL injection**: a value typed by a user that contains SQL of its own becomes part of the statement, and a crafted name can read or destroy the table. Second, correctness: the database driver quotes and escapes each value for you, so a name containing an apostrophe, or a number that arrives as text, is handled properly instead of breaking the statement.

**(d) commit, fetchone and fetchall.** `commit()` makes the changes since the last commit permanent. Insert, update or delete without it and the work is **rolled back and lost** when the connection closes, with no error printed, which is why a program can look correct and save nothing.

**What it returns**

**When to use it**

|                         |                                                                                  |                                                                        |
|-------------------------|----------------------------------------------------------------------------------|------------------------------------------------------------------------|
| <code>fetchone()</code> | The NEXT single row, as a tuple, or None when there are no more                  | When the query can only match one row, such as a lookup by primary key |
| <code>fetchall()</code> | A LIST of every remaining row, each a tuple, and an empty list if there are none | When you want them all, to loop over or count                          |

```
import sqlite3

conn = sqlite3.connect(":memory:")
cur = conn.cursor()
cur.execute("CREATE TABLE students (name TEXT, score REAL)")
cur.executemany("INSERT INTO students VALUES (?, ?)", [("Ada", 72.5), ("Grace", 64.0)])
conn.commit()

cur.execute("SELECT * FROM students")
print("fetchone:", cur.fetchone())
print("fetchall of what is left:", cur.fetchall())
print("fetchall again:", cur.fetchall())
conn.close()
```

```
fetchone: ('Ada', 72.5)
fetchall of what is left: [('Grace', 64.0)]
fetchall again: []
```

The last line is the detail worth knowing: a cursor is consumed as it is read, so a second fetch returns nothing until the query is run again. `sqlite3.connect(":memory:")`, used here, makes a database that lives only in memory, which is handy for testing.

## Question Two: tkinter: a window, its widgets and its layout

### BREAK IT DOWN FIRST

before you write a word

Which one line, left out, makes the window flash up and vanish? And how does a button run your function without calling it?

#### ANSWER

**The line** `window.mainloop()`. Without it the program reaches its end and exits, taking the window with it.

**The button** `command=convert`, with NO brackets. Brackets would call the function immediately, once, at start up, and pass its return value to the button. The name alone hands the function itself over to be called on each click.

**Reading the box** `entry.get()` returns a STRING, like input, so it must be converted before arithmetic.

**Showing the answer** A Label's text is changed with `label.config(text=...)`, or by using a `StringVar`.

## PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

(a) GUI, tkinter, Tk() and mainloop(). (b) Five widgets. (c) The three geometry managers and the rule about mixing them. (d) A tkinter naira to dollars converter.

**(a) The terms.** A **GUI**, a graphical user interface, lets a user work with a program through windows, buttons and boxes rather than by typing at a prompt. **tkinter** is Python's standard GUI library, which ships with Python and wraps the Tk toolkit.

- `window = Tk()` creates the **main window**, the top level container everything else is placed inside. It is the first tkinter line in any program.
- `window.mainloop()` starts the **event loop**: the program stops running top to bottom and instead waits for events, clicks, key presses, window closes, dispatching each to whatever handles it. It is the LAST line, and without it the program ends immediately and the window disappears.

**(b) Five widgets.**

### Widget

### What it is for

**Label** Displays text or an image that the user cannot edit. Used for prompts and results

**Entry** A single line box the user types into. `get()` reads it, as a string

**Button** A clickable button. Its `command` names the function to run on each click

**Text** A multi line editable text area, for anything longer than one line

**Frame** A container that groups other widgets, so a window can be laid out in sections

Others that earn the mark: **Checkbutton**, **Radiobutton**, **Listbox**, **Scale**, **Canvas**, **Menu**, **Messagebox**.

**(c) The three geometry managers.**

### Manager

### How it places a widget

`pack()` Stacks widgets in a block, against a side, top by default. Quickest for a simple column of widgets

`grid()` Places widgets in rows and columns, by number. Best for a form, where labels and boxes must line up

`place()` Places a widget at exact x and y coordinates. Precise, and it does not adapt when the window is resized

**The rule:** pack and grid must **not be mixed inside the same container**. Doing so makes the program hang while the two managers argue over the same space. Use one per container; different frames may each use a different one.

**(d) The converter.** This listing is not run here, because its output is a window rather than text.

converter.py

```

from tkinter import Tk, Label, Entry, Button

RATE = 1650

def convert():
    try:
        naira = float(entry.get())
    except ValueError:
        result.config(text="Enter a number")
    return
    dollars = naira / RATE
    result.config(text=f"{naira:,.2f} naira is {dollars:,.2f} dollars")

window = Tk()
window.title("Naira to dollars")

Label(window, text="Amount in naira:").grid(row=0, column=0, padx=8, pady=8)

entry = Entry(window, width=18)
entry.grid(row=0, column=1, padx=8, pady=8)

Button(window, text="Convert", command=convert).grid(row=1, column=0, columnspan=2, pa

result = Label(window, text="")
result.grid(row=2, column=0, columnspan=2, padx=8, pady=8)

window.mainloop()

```

Five things the examiner is looking for: Tk() first and mainloop() last; command=convert with **no brackets**, so the function is handed over rather than called at start up; entry.get() converted with float, because it returns a string; the result shown by changing a Label with config; and one geometry manager throughout, grid here, because the form has two columns. The try and except around the conversion is what stops a letter typed in the box from killing the program.

## The 2025/2026 Paper, Every Question

Past theory questions

## How the Paper Changed

## Questions One to Three, Worked

AS PRINTED

2025/2026 paper, Question One (a)



Briefly describe how the `and` operator works in Python (use examples) [2.5 marks]

**BREAK IT DOWN FIRST**

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

**ANSWER**

What I am given

Nothing but the operator name. This is a bookwork part: no data, no program, just **describe** and **show**.

What is actually asked

Two things joined by "(use examples)". A description with no example cannot score full marks, and an example with no description cannot either. Answer both halves explicitly.

What the marks say

2.5 marks is the smallest part on the paper. A truth table plus one worked line is the right size. Do not write an essay here and run out of time on the 10.5-mark database question.

Where it is taught

Module Two, Unit 3 (Operators).

**MODEL ANSWER**

2025/2026 paper, Question One (a)

Write your answer to **2025/2026 paper, Question One (a)** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

**ANSWER · Q1 (A) THE AND OPERATOR, 2.5 MARKS**

**and** is a boolean operator. It joins two conditions and is true **only when both are true**. If either side is false, the whole thing is false.

| Left  | Right | left and right |
|-------|-------|----------------|
| True  | True  | True           |
| True  | False | False          |
| False | True  | False          |
| False | False | False          |

The example the marker wants is a range check:

```
>>> score = 72
>>> score >= 0 and score <= 100
True
```

Both sides hold, so the score is valid. Change score to 150 and the right side fails, so the whole test is False. and also short-circuits: if the left is false, Python never even looks at the right, because the answer is already settled.

## AS PRINTED

2025/2026 paper, Question One (b)

THE EXAMINER'S WORDS, UNEDITED

What is a string in Python? List and briefly explain four common string methods used in Python. Provide an example for each method. [5 marks]

## BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

### ANSWER

What is actually asked

Three instructions in one part: **define**, **list and explain four**, and **give an example for each**. That is nine separate things to write.

What the marks say

5 marks over roughly nine items means each is worth about half a mark. Short and correct beats long and vague. A table is the fastest way to earn all of them.

The trap

It says **four**. Writing six wastes time and earns nothing extra; writing three throws marks away. Count what you wrote.

Where it is taught

Module Two, Unit 4 (Working with Strings).

## MODEL ANSWER

2025/2026 paper, Question One (b)

Write your answer to **2025/2026 paper, Question One (b)** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

ANSWER · Q1 (B) STRINGS AND FOUR METHODS, 5 MARKS

A **string** is an ordered sequence of characters, written in quotes. It is **immutable**: its characters cannot be changed in place. Four common methods, one example each:

| Method  | Does       | Example        | Gives |
|---------|------------|----------------|-------|
| upper() | Capitals   | "hi".upper()   | 'HI'  |
| lower() | Lower case | "Hi".lower()   | 'hi'  |
| strip() | Trims ends | " hi ".strip() | 'hi'  |

|              |            |                       |      |
|--------------|------------|-----------------------|------|
| replace(a,b) | Swaps text | "hi".replace("i","o") | 'ho' |
|--------------|------------|-----------------------|------|

Each returns a **new** string, because the original cannot change. Others you know and could name instead: `split()`, `count()`, `find()`, `title()`. See Module Two, Unit Four for all of them.

## AS PRINTED

2025/2026 paper, Question One (c)

THE EXAMINER'S WORDS, UNEDITED

Identify the Errors in the following code segment [3 marks]

```
1 person = { "name": 'John Doe', 'age': 30, 'city':  
2     'New York', 'Email': 'john.doe@example.com' }  
3  
4 print("Name:", person['Name'])  
5 print("Age:", person['Age'])  
6 print("City:", person['City'])  
7 print("Email:", person['email'])  
8  
9 person['Age'] = "35"  
10 person['phone'] = '123-456-7890'  
11 del person['email']  
12  
13 print("Updated Person Dictionary:")  
14 print(person)  
15
```

## BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

### ANSWER

What is actually asked

**Identify** only. It does not ask you to rewrite the code, unlike Question Two (c), which does. Listing the faults earns the marks here; a full corrected listing earns no more.

What I am given

A dictionary whose keys are 'name', 'age', 'city' and 'Email', then six accesses to it.

Where the trap sits

Every fault is the **same** fault: a dictionary key is case sensitive. Do not hunt for a syntax error, there is none. The line that raises nothing at all is the most dangerous one on the page.

What the marks say

3 marks and there are more than three faults, so name them in groups rather than one line at a time.

Where it is taught

Module Three, Unit 4 (Sets and Dictionaries).

## MODEL ANSWER

2025/2026 paper, Question One (c)

Write your answer to **2025/2026 paper, Question One (c)** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

### ANSWER · Q1 (C) THE DICTIONARY ERRORS, 3 MARKS

The dictionary defines its keys as 'name', 'age', 'city' and 'Email'. Every access then uses the **wrong case**, and a dictionary key is case sensitive, so each is a `KeyError`. Python stops at the first one it reaches:

```
Traceback (most recent call last):
  File "person.py", line 4, in <module>
    print("Name:", person['Name'])
KeyError: 'Name'
```

The full list of faults, so you earn every mark:

- `person['Name']`, `person['Age']`, `person['City']`: the keys are lower case. Three `KeyErrors`.
- `person['email']` on line 7 and `del person['email']` on line 11: the key is 'Email' with a capital E.
- `person['Age'] = "35"` does not raise, but it **creates a new key** 'Age' beside the real 'age', rather than updating the age. A silent logic error.

Corrected so the case matches throughout, this is what it prints:

person.py, corrected

```
person = {"name": "John Doe", "age": 30,
          "city": "New York", "Email": "john.doe@example.com"}
print("Name:", person["name"])
print("Age:", person["age"])
print("City:", person["city"])
print("Email:", person["Email"])
person["age"] = "35"
person["phone"] = "123-456-7890"
del person["Email"]
print("Updated Person Dictionary:")
print(person)
```

Name: John Doe

Age: 30

City: New York

Email: john.doe@example.com

Updated Person Dictionary:

```
{'name': 'John Doe', 'age': '35', 'city': 'New York', 'phone': '123-456-7890'}
```

Note the final dictionary: 'age' is now the string '35', 'phone' was added, and 'Email' was deleted. That is del from Module Three doing its job.

### AS PRINTED

2025/2026 paper, Question One (d)

THE EXAMINER'S WORDS, UNEDITED

Write a Python program that implements a mathematical model to calculate Body Mass Index (BMI). Use the formula:  $BMI = W / H^2$ , where: W = weight in kilograms, H = height in meters. [3.5 marks]

- 1. Classify the BMI:
- $BMI < 18.5 \rightarrow \text{Underweight}$
- $18.5 \leq BMI < 25 \rightarrow \text{Normal weight}$
- $25 \leq BMI < 30 \rightarrow \text{Overweight}$
- $BMI \geq 30 \rightarrow \text{Obese}$
- 2. Display the BMI value with the BMI class.

### BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

#### ANSWER

What I am given

The formula and four bands, with their boundaries written as  $\leq$  and  $<$ . The paper has done the hard thinking; copy its boundaries exactly rather than inventing your own.

What is actually asked

A **program**, not a function and not a formula. It must read input, compute, classify and print.

Where the trap sits

The bands touch. 25 is Overweight, not Normal weight, because the second band stops **below** 25.

An if/elif chain in ascending order gets this right for free; a chain of separate ifs does not.

Where it is taught

Module Three, Unit 1 (Decision Making).

## MODEL ANSWER

2025/2026 paper, Question One (d)

Write your answer to **2025/2026 paper, Question One (d)** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

**ANSWER · Q1 (D) BMI, 3.5 MARKS**

$H^2$  in the question is `H ** 2` in Python: `**` is the power operator from Module Two, not `^`, which is something else entirely. Read the two values, compute, then classify with if/elif.

bmi.py

```
weight = float(input("Weight (kg): "))
height = float(input("Height (m): "))

bmi = weight / height ** 2

if bmi < 18.5:
    category = "Underweight"
elif bmi < 25:
    category = "Normal weight"
elif bmi < 30:
    category = "Overweight"
else:
    category = "Obese"

print(f"BMI = {bmi:.2f}")
print("Class:", category)
```

For a 72 kg person 1.75 m tall:

BMI = 23.51

Class: Normal weight

The elif ladder needs no upper bound on each rung: by the time you reach `elif bmi < 25`, the first if has already ruled out everything below 18.5, so `< 25` alone means the 18.5 to 25 band. That is the whole economy of elif.

## AS PRINTED

2025/2026 paper, Question One (e)

THE EXAMINER'S WORDS, UNEDITED

Write a Python program that calculates the future value of an investment using compound interest. The program should display A and the interest earned (A - P) to two decimal places. [3.5 marks]

- -P = principal
- -r = annual interest rate (decimal)
- -n = number of times interest is compounded per year
- -t = time in years

Use the formula:  $A = P(1 + r/n)^{nt}$  where:

### BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

#### ANSWER

What is actually asked

Two printed values, not one: **A** and **the interest earned**, both to two decimal places. Printing only A loses marks.

Reading the formula

^ in the paper means "to the power of". In Python that is \*\*, never ^, which is a different operator entirely.

Where the trap sits

"(decimal)" beside r means 0.05, not 5. And "to two decimal places" is a formatting instruction: use an f-string with :.2f, do not round the stored value.

Where it is taught

Module Two, Unit 3 (Operators) and Unit 4 (formatting).

### MODEL ANSWER

2025/2026 paper, Question One (e)

Write your answer to **2025/2026 paper, Question One (e)** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

**ANSWER · Q1 (E) COMPOUND INTEREST, 3.5 MARKS**

The formula  $A = P(1 + r/n)^{(nt)}$  is one line once you use `**` for the power and brackets for the exponent, so the whole  $n \cdot t$  is the power:

interest.py

```
P = float(input("Principal: "))
r = float(input("Annual rate (decimal): "))
n = int(input("Times compounded per year: "))
t = int(input("Years: "))

A = P * (1 + r / n) ** (n * t)

print(f"Future value (A): {A:.2f}")
print(f"Interest earned: {A - P:.2f}")
```

For 100000 at 0.05, compounded 4 times a year for 3 years:

Future value (A): 116075.45

Interest earned: 16075.45

The `:.2f` rounds to two decimal places for display only. The mark hinges on the brackets: `** (n * t)`, not `** n * t`, which would raise to the power  $n$  and only then multiply by  $t$ , a different and wrong number.

**AS PRINTED**

2025/2026 paper, Question Two (a)

THE EXAMINER'S WORDS, UNEDITED

What is a count-controlled loop in Python? [3.5 mark]

**BREAK IT DOWN FIRST**

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

**ANSWER**

What is actually asked

A definition. One clear sentence plus a short example is the whole answer.

Note the paper

The mark text reads **3.5 mark**, singular, where every other part says "marks". That is the paper's own slip and it is quoted here as printed. It means 3.5 marks.

What the marks say



3.5 marks for a definition is generous, so do not stop at one line: name the loop, say what controls it, and show a for with range().

Where it is taught

Module Three, Unit 2 (Loops).

### MODEL ANSWER

2025/2026 paper, Question Two (a)

Write your answer to **2025/2026 paper, Question Two (a)** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

**ANSWER · Q2 (A) COUNT-CONTROLLED LOOP, 3.5 MARKS**

A **count-controlled loop** runs a known, fixed number of times, once per item in a sequence. In Python it is the for loop with range. You use it when you know how many passes you need before the loop starts.

```
for i in range(5):  
    print(i)
```

It contrasts with a **condition-controlled loop** (while), which runs an unknown number of times, until a condition turns false. Count-controlled: "do this five times". Condition-controlled: "do this until the user types done".

### AS PRINTED

2025/2026 paper, Question Two (b)

THE EXAMINER'S WORDS, UNEDITED

Explain the differences among the main types of control structures in Python.  
Give one example of each. [4.5 marks]

### BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

**ANSWER**

What is actually asked

"Differences **among**" means compare them against each other, not define each in isolation. Then one example each.

What I should name

Sequence, selection and repetition. Naming only if and for misses a third of the answer.

What the marks say

4.5 marks across three structures, each needing a difference and an example, is 1.5 a structure. Keep each tight.

Where it is taught

Module Three, Units 1 and 2.

### MODEL ANSWER

2025/2026 paper, Question Two (b)

Write your answer to **2025/2026 paper, Question Two (b)** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

#### ANSWER · Q2 (B) CONTROL STRUCTURES, 4.5 MARKS

There are three kinds of control structure, and the answer names all three with an example each:

- **Sequence:** statements run top to bottom, in order. The default. `a = 2; b = 3; print(a + b)`
- **Selection:** a choice, with `if/elif/else`. `if score >= 50: print("Pass")`
- **Repetition:** doing something many times, with `for` or `while`. `for i in range(3): print(i)`

The difference is what decides the flow: sequence has no decision, selection runs a block or skips it once, repetition runs a block again and again.

### AS PRINTED

2025/2026 paper, Question Two (c)

#### THE EXAMINER'S WORDS, UNEDITED

Identify the Errors in this code segment and write the corrected version. [3 marks]

```
1 my_string = "Serve Jesus Now"
2 print("Original String:", mystring)
3
4 first_character = my_string[1]
5 last_character = my_string[-0]
6 print("First Character:", first_character)
7 print("Last Character:" last_character)
8
9 substring = my_string[17:]
10 print("Substring:", substring)
11
12 greeting = "Hi"
13 name = "Alice"
14 message = greeting + ", " + name + "!"
15 print("Message:", message)
16
17 length = len(mystring)
18 print("Length of the String:", length)
19
```

```
20 count_e = my_string.count('e')
21 print("Count of 'e':", count_e)
22
```

## BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

### ANSWER

What is actually asked

Two things, and this is the difference from Question One (c): identify the errors **and write the corrected version**. A list of faults alone is half an answer here.

Where the traps sit

Three of the faults stop the program and two are silent. Line 1 never closes its string, so nothing runs at all until that is fixed. Line 5 uses `[-0]`, which is `[0]`, so it quietly returns the **first** character while claiming to be the last.

Order of discovery

Fix the syntax first, then run it in your head again. A `SyntaxError` hides every fault after it, so the name errors on lines 2 and 17 cannot even appear until line 1 is closed.

Where it is taught

Module Two, Unit 4 (Strings) and Module Four, Unit 4 (Exceptions).

## MODEL ANSWER

2025/2026 paper, Question Two (c)

Write your answer to **2025/2026 paper, Question Two (c)** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

### ANSWER · Q2 (C) STRING ERRORS, 3 MARKS

The variable is `my_string`, but lines 2 and 17 read `mystring`, which was never defined. Line 2 raises first:

```
NameError: name 'mystring' is not defined
```

Two more are logic faults that run without complaint but give the wrong thing:

- `my_string[1]` is the **second** character, not the first. The first is `[0]`.
- `my_string[-0]` is `my_string[0]`, because `-0` is `0`. It gives the first character, not the last. The last is `[-1]`.
- `my_string[17:1]` starts past the end and runs backwards, so it is the empty string. A real substring needs `start < stop` within the length.

Corrected, taking the first word "Serve" as the substring:

my\_string.py, corrected

```
my_string = "Serve Jesus Now"
print("Original String:", my_string)
first_character = my_string[0]
last_character = my_string[-1]
print("First Character:", first_character)
print("Last Character:", last_character)
substring = my_string[6:11]
print("Substring:", substring)
greeting = "Hi"
name = "Alice"
message = greeting + ", " + name + "!"
print("Message:", message)
length = len(my_string)
print("Length of the String:", length)
count_e = my_string.count("e")
print("Count of 'e':", count_e)
```

Original String: Serve Jesus Now  
First Character: S  
Last Character: w  
Substring: Jesus  
Message: Hi, Alice!  
Length of the String: 15  
Count of 'e': 3

## AS PRINTED

2025/2026 paper, Question Two (d)

### THE EXAMINER'S WORDS, UNEDITED

A secondary school is developing a simple Python program to assist teachers in managing students' assessment activities. The school has provided the following grading scheme: [6.5 marks]

- -70 – 100: Grade A
- -60 – 69: Grade B
- -50 – 59: Grade C

- -45 – 49: Grade D
- -0 – 44: Grade F

Given that the correct password for accessing the system is: admin123, write a Python program that uses appropriate control structures to perform the required tasks.

### BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

#### ANSWER

What is actually asked

One program doing **four** separate jobs: a password loop, a validated score with a grade, a multiplication table, and a sentinel loop that totals numbers. Read all eight requirements before you start, because they are really four programs in a row.

What the marks say

6.5 marks over eight requirements. Each requirement is worth under a mark, so a part-finished program still scores well: write the four blocks in order and do not stall on the first.

Where the traps sit

Requirement i says **continue requesting until** correct, which is a **while**, not an **if**. Requirement viii excludes the terminating 0 from the sum **and** from the count.

Where it is taught

Module Three, Units 1 and 2. The sentinel loop is the idiom drilled in both mocks.

### MODEL ANSWER

2025/2026 paper, Question Two (d)

Write your answer to **2025/2026 paper, Question Two (d)** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

#### ANSWER · Q2 (D) THE CONTROL-STRUCTURES PROGRAM, 6.5 MARKS

Six tasks in one program. Take them in order: a password loop, a validated score, a grade, a multiplication table, then a running sum. Each is a control structure from Module Three.

system.py

```
# i. password loop
while True:
    pw = input("Password: ")
```

```

if pw == "admin123":
    print("Access Granted")
    break
print("Access Denied")

# ii and iii. read a score and validate it
score = int(input("Score: "))
if score < 0 or score > 100:
    print("Error: score must be between 0 and 100")
else:
    # iv. grade it
    if score >= 70:
        print("Grade A")
    elif score >= 60:
        print("Grade B")
    elif score >= 50:
        print("Grade C")
    elif score >= 45:
        print("Grade D")
    else:
        print("Grade F")

# v. multiplication table 1 to 12
num = int(input("Number: "))
for i in range(1, 13):
    print(f"{num} x {i} = {num * i}")

# vi. sum until 0
total = 0
count = 0
while True:
    n = int(input("Number (0 to stop): "))
    if n == 0:
        break
    total += n
    count += 1
print("Sum:", total)
print("Count:", count)

```

The grade ladder is the same elif economy as the BMI: each rung needs only its lower bound, because the rungs above have already been ruled out. The order matters: put `>= 45` before `>= 70` and every score would grade D. Ranges must be tested high to low.

The two `while True` loops are the two loop patterns from Module Three, Unit Two: one broken by a right password, one by a sentinel of 0.

**AS PRINTED**

2025/2026 paper, Question Three (a)

THE EXAMINER'S WORDS, UNEDITED

Mention five (5) benefits of modularizing a program with functions [2.5 marks]

### BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

#### ANSWER

What is actually asked

"Mention" means list. No explanation is demanded, so five short named benefits is a complete answer.

What the marks say

2.5 marks over five items is half a mark each. Write exactly five: a sixth earns nothing, and four costs you half a mark.

Where it is taught

Module Four, Unit 1 (Functions).

### MODEL ANSWER

2025/2026 paper, Question Three (a)

Write your answer to **2025/2026 paper, Question Three (a)** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

#### ANSWER · Q3 (A) FIVE BENEFITS, 2.5 MARKS

Five benefits of modularizing a program with functions, which is building it out of named pieces:

1. **Reuse:** write it once, call it anywhere, no copying.
2. **Readability:** a name says what a block does.
3. **Maintenance:** fix a bug in one place and every caller is fixed.
4. **Decomposition:** a large problem splits into small, testable pieces.
5. **Abstraction:** a caller uses a function knowing what it does, not how.

Module Four, Unit One carries the same list under "why functions matter".

### AS PRINTED

2025/2026 paper, Question Three (b)

#### THE EXAMINER'S WORDS, UNEDITED

What is a function in Python? Why are functions important in programming?  
Differentiate between built-in functions and user-defined functions. Distinguish between a Void and Value-returning function in Python (with examples). [6 marks]

## BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

### ANSWER

What is actually asked

Four questions in one part: define, justify, differentiate built-in from user-defined, and distinguish void from value-returning. Answer them as four labelled paragraphs so the marker can find all four.

What the marks say

6 marks, so 1.5 per question. The "(with examples)" attaches to the last one and is not optional.

Why this part matters

Void versus value-returning comes back twice more on this same paper, in Question Six (b) and (c), worth 14 marks between them. Getting the distinction crisp here pays three times.

Where it is taught

Module Four, Unit 1.

## MODEL ANSWER

2025/2026 paper, Question Three (b)

Write your answer to **2025/2026 paper, Question Three (b)** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

### ANSWER · Q3 (B) FUNCTIONS, BUILT-IN VS USER-DEFINED, 6 MARKS

A **function** is a named block of code that runs only when you call it. They matter for the five reasons above. The two kinds:

|                  | Built-in          | User-defined            |
|------------------|-------------------|-------------------------|
| Who wrote it     | Comes with Python | You did, with def       |
| Defining needed? | No, always there  | Yes, before you call it |
| Examples         | print, len, input | cube, grade_of          |

`print("hi")` calls a built-in; after `def cube(n): return n*n*n` you call your own with `cube(4)`.

## AS PRINTED

2025/2026 paper, Question Three (c)

### THE EXAMINER'S WORDS, UNEDITED

What will `list1` and `list2` contain after the following statements are executed? [3 marks]

- `i.list1 = [1, 2] * 2`



- `ii.list2 = [3]`
- `iii.list2 += list1`

### BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

#### ANSWER

What is actually asked

The **contents of both lists** at the end, not a trace of each line. Write the two final values.

Where the trap sits

\* on a list repeats it, it does not multiply the numbers inside. And += on a list extends it in place rather than nesting, so the result is flat.

What the marks say

3 marks for two answers means the working counts. Show the intermediate value of list1.

Where it is taught

Module Three, Unit 3 (Lists and Tuples).

### MODEL ANSWER

2025/2026 paper, Question Three (c)

Write your answer to **2025/2026 paper, Question Three (c)** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

#### ANSWER · Q3 (C) LIST1 AND LIST2, 3 MARKS

```
list1 = [1, 2] * 2
list2 = [3]
list2 += list1
```

- `[1, 2] * 2` **repeats the list**, so list1 is `[1, 2, 1, 2]`, not `[2, 4]`.
- `list2 += list1` extends list2 in place, tacking on every item of list1.

```
list1 = [1, 2, 1, 2]
list2 = [3, 1, 2, 1, 2]
```

The trap is reading \* as multiply-the-numbers. On a list it repeats. See Module Three, Unit Three.

## THE EXAMINER'S WORDS, UNEDITED

The Computer Science Department organized a beginner Python Programming Workshop for undergraduate students. You have been asked to write a simple Python program to manage the workshop information. The organizers provided the following records:

- -Registered Students: Esther, John, Paula, Paul, Peter, Ada, John, Mary
- -Workshop Details: "Python Programming Workshop", "Room 204", "Monday", 30
- -Students Who Attended: Esther, John, Mercy, Paul, Esther, John, Paula
- -Student Scores:
- -Esther scored 60
- -John scored 55
- -Mercy scored 65
- -Paul scored 80

Write a Python program that uses the most suitable Python collection for each type of data. [6 marks]

## BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

## ANSWER

What is actually asked

The phrase that decides the whole question is **the most suitable Python collection for each type of data**. The marks are for **choosing**, not just for printing.

Reading the data

The names repeat: John appears twice in the registered list, Esther and John twice in the attendance list. That repetition is not a typo, it is the question telling you which collection to reach for.

What the marks say

6 marks and ten requirements, so no single line is worth much. Requirements viii and ix are one operator each once the right collection is chosen.

Where it is taught

Module Three, Units 3 and 4.

## MODEL ANSWER

2025/2026 paper, Question Three (d)

Write your answer to **2025/2026 paper, Question Three (d)** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

**ANSWER · Q3 (D) THE COLLECTIONS PROGRAM, 5 MARKS**

Each kind of data wants a different collection, and choosing right **is** the question:

- Registered students, who may repeat and whose order does not matter, but which you want **unique**: a **set**.
- Workshop details, four fixed facts that never change: a **tuple**.
- Attendance, with duplicates to remove: a **set**.
- Scores, a name against a number: a **dictionary**.

The two "registered but did not attend" and "attended but not registered" questions are **set difference** both ways, from Module Three, Unit Four.

workshop.py

```
registered = {"Esther", "John", "Paula", "Paul",
              "Peter", "Ada", "Mary"}
details = ("Python Programming Workshop", "Room 204", "Monday", 30)
attended = {"Esther", "John", "Mercy", "Paul", "Paula"}
scores = {"Esther": 72, "John": 55, "Paula": 68,
          "Paul": 40, "Peter": 91, "Ada": 59, "Mary": 83}

print("Workshop details:", details)
print("Registered:", sorted(registered))
print("Attended:", sorted(attended))
print("Registered, did not attend:", sorted(registered - attended))
print("Attended, not registered:", sorted(attended - registered))
print("Scored 60+:", sorted(n for n, s in scores.items() if s >= 60))
```

```
Workshop details: ('Python Programming Workshop', 'Room 204', 'Monday', 30)
Registered: ['Ada', 'Esther', 'John', 'Mary', 'Paul', 'Paula', 'Peter']
Attended: ['Esther', 'John', 'Mercy', 'Paul', 'Paula']
Registered, did not attend: ['Ada', 'Mary', 'Peter']
Attended, not registered: ['Mercy']
Scored 60+: ['Esther', 'Mary', 'Paula', 'Peter']
```

**Why sorted is around every set:** a set has no order, so its printed order is not reliable. Wrapping it in sorted gives a stable, markable list every run. Mercy attended without registering; Ada, Mary and Peter registered but did not come.

## Questions Four to Six, Worked

### AS PRINTED

2025/2026 paper, Question Four (a)

THE EXAMINER'S WORDS, UNEDITED

Strings are Immutable. Explain and with reasons. [5 marks]

### BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

#### ANSWER

Note the paper

It reads "Explain and with reasons", which is not quite grammatical. It is the paper's own wording, quoted as printed. Read it as "explain, and give reasons".

What is actually asked

Not just what immutable means, but **why**. Reasons carry the marks: an explanation with no reason is half an answer.

What the marks say

5 marks is a lot for a bookwork part, so show it as well as saying it: a two-line demonstration that reassignment makes a new string is worth more than another paragraph.

Where it is taught

Module Two, Unit 4.

### MODEL ANSWER

2025/2026 paper, Question Four (a)

Write your answer to **2025/2026 paper, Question Four (a)** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

**ANSWER · Q4 (A) STRINGS ARE IMMUTABLE**

**Immutable** means a string cannot be changed after it is made. Trying to change one character is an error:

```
>>> s = "cat"
>>> s[0] = "b"
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'str' object does not support item assignment
```

**The reason:** every method that "changes" a string, like `upper()` or `replace()`, actually returns a **new** string and leaves the original untouched. To "edit" a string you build a new one and reassign it: `s = s.replace("c", "b")`. A list, by contrast, is mutable and does allow `list[0] = "b"`.

## AS PRINTED

2025/2026 paper, Question Four (b)

THE EXAMINER'S WORDS, UNEDITED

A program contains the following function definition:

```
2
3 def cube(num):
4     return num * num * num
5
```

- i. Write a statement that passes the value 4 to this function and assigns its return value to the variable `result`. [1 mark]
- ii. What will be the value in `result` [1 mark]

## BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

### ANSWER

What is actually asked

One line of code, then one number. Nothing more: part i is a single statement and part ii is a single value.

Note the paper

Part ii is printed without a question mark. Quoted as printed; it is still a question.

What the marks say

1 mark each, the smallest on the paper. Write the line, write the number, move on.

Where it is taught

Module Four, Unit 1.

## MODEL ANSWER

2025/2026 paper, Question Four (b)

Write your answer to **2025/2026 paper, Question Four (b)** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

**ANSWER · Q4 (B) THE CUBE FUNCTION, 1 + 1 MARKS**

(i) A statement that passes 4 and stores the result:

```
result = cube(4)
```

(ii) `cube(4)` returns  $4 * 4 * 4$ , so:

```
result = 64
```

The call `cube(4)` **is** a value, because `cube` returns one, so it can sit on the right of an `=`. That is a value-returning function from Module Four.

## AS PRINTED

2025/2026 paper, Question Four (c)

THE EXAMINER'S WORDS, UNEDITED

A secondary school has decided to computerize the management of students' academic records. You have been asked to develop a simple Python program that uses an SQLite database to store, retrieve, update, and manage students' records. The school requires that all records be stored in a database named `school.db` in a table named `student`. The table should contain the following fields: `StudentID` (INTEGER), `StudentName` (TEXT), `Class` (TEXT), and `Score` (REAL). The school has provided the following student records: [10.5 marks]

| StudentID | StudentName | Class | Score |
|-----------|-------------|-------|-------|
|-----------|-------------|-------|-------|

|     |             |     |    |
|-----|-------------|-----|----|
| 101 | David James | SS1 | 78 |
|-----|-------------|-----|----|

- 102 | Mary Johnson | SS2 | 85
- 103 | Peter Okoro | SS1 | 67
- 104 | Grace Bello | SS3 | 92
- 105 | Esther Ade | SS2 | 74

Write a Python program that uses SQLite to manage the students' records.

### BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

#### ANSWER

What is actually asked

The single biggest part on the paper. Seven requirements: create, insert, read all, read filtered, aggregate, update, delete. That is the whole of SQL in one question.

What the marks say

10.5 marks is three of the small parts put together, and it is 15 per cent of the whole examination. If you can do this question, attempt it first: nothing else on the paper pays as well per line written.

Where the traps sit

The names are exact: database `school.db`, table `student` (singular), and those four field names with those four types. Marks go for matching them, not for a working program with your own names. Requirement i says **if they do not already exist**.

Where it is taught

Module Five, Unit 1 (Python and Databases).

### MODEL ANSWER

2025/2026 paper, Question Four (c)

Write your answer to **2025/2026 paper, Question Four (c)** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

#### ANSWER · Q4 (C) THE SQLITE PROGRAM, 10.5 MARKS

The biggest single part on the paper, and it walks through the whole of Module Five, Unit One plus the new WHERE, UPDATE, DELETE and summary functions. Take the seven requirements in order.

```

import sqlite3

conn = sqlite3.connect("school.db")
cursor = conn.cursor()

# i. create the table if it is not there
cursor.execute("""CREATE TABLE IF NOT EXISTS student
                  (StudentID INTEGER PRIMARY KEY, StudentName TEXT,
                   Class TEXT, Score REAL)""")

# ii. insert the given records
records = [(101, "David James", "SS1", 78),
           (102, "Mary Johnson", "SS2", 85),
           (103, "Peter Okoro", "SS1", 67),
           (104, "Grace Bello", "SS3", 92),
           (105, "Esther Ade", "SS2", 74)]
cursor.executemany("INSERT INTO student VALUES (?, ?, ?, ?)", records)
conn.commit()

# iii. all records
print("All students:")
cursor.execute("SELECT * FROM student")
for row in cursor.fetchall():
    print(row)

# iv. SS2 only
print("SS2 students:")
cursor.execute("SELECT * FROM student WHERE Class = ?", ("SS2",))
for row in cursor.fetchall():
    print(row)

# v. statistics in one query
cursor.execute("SELECT COUNT(*), MAX(Score), AVG(Score) FROM student")
total, highest, average = cursor.fetchone()
print("Total:", total, " Highest:", highest, " Average:", average)

# vi. update Peter's score
cursor.execute("UPDATE student SET Score = ? WHERE StudentName = ?",
               (75, "Peter Okoro"))
conn.commit()

# vii. delete student 105
cursor.execute("DELETE FROM student WHERE StudentID = ?", (105,))
conn.commit()

conn.close()

```

The output of the two SELECTs and the statistics:



All students:

```
(101, 'David James', 'SS1', 78.0)
(102, 'Mary Johnson', 'SS2', 85.0)
(103, 'Peter Okoro', 'SS1', 67.0)
(104, 'Grace Bello', 'SS3', 92.0)
(105, 'Esther Ade', 'SS2', 74.0)
```

SS2 students:

```
(102, 'Mary Johnson', 'SS2', 85.0)
(105, 'Esther Ade', 'SS2', 74.0)
```

Total: 5 Highest: 92.0 Average: 79.2

Five marks are in the details a rushed answer drops:

- **Scores print as 78.0**, with a decimal, because the column is REAL.
- **Placeholders, not glued strings**, for every value. ("SS2",) keeps its comma.
- **UPDATE and DELETE each commit**. Forget the commit and the change is thrown away silently.
- **The statistics are one query**. COUNT, MAX and AVG come back as a single row, unpacked into three names.

#### AS PRINTED

2025/2026 paper, Question Five (a)

THE EXAMINER'S WORDS, UNEDITED

What is file handling in Python? [2 marks]

#### BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

#### ANSWER

What is actually asked

A definition. Two marks, two sentences.

Where it is taught

Module Four, Unit 3 (File Handling).

#### MODEL ANSWER

2025/2026 paper, Question Five (a)

Write your answer to **2025/2026 paper, Question Five (a)** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

ANSWER · Q5 (A) FILE HANDLING, 2 MARKS

**File handling** is reading data from, and writing data to, a file on disk, so the data lasts after the program ends. Python does it with `open()`, the read and write methods, and `close()` (or a `with`

block that closes for you). See Module Four, Unit Three.

## AS PRINTED

2025/2026 paper, Question Five (b)

THE EXAMINER'S WORDS, UNEDITED

List and explain four (4) file access modes used in the open() function. [6 marks]

## BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

### ANSWER

What is actually asked

Four modes, each **listed and explained**. The mode letter alone is not an explanation.

What the marks say

6 marks over four modes is 1.5 each, so each needs the letter, what it does, and what happens if the file does not exist.

Where the trap sits

The difference that earns the marks is what each mode does to an **existing** file. One of them silently destroys it.

Where it is taught

Module Four, Unit 3.

## MODEL ANSWER

2025/2026 paper, Question Five (b)

Write your answer to **2025/2026 paper, Question Five (b)** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

ANSWER · Q5 (B) FOUR FILE MODES, 6 MARKS

| Mode | Means          | If the file exists  | If it does not    |
|------|----------------|---------------------|-------------------|
| "r"  | Read           | Reads it            | FileNotFoundError |
| "w"  | Write          | <b>Erased first</b> | Created           |
| "a"  | Append         | Adds at the end     | Created           |
| "r+" | Read and write | Both, no erase      | FileNotFoundError |

The two that catch people: "w" **destroys** the existing contents the moment you open it, and "r" is the only common one that fails on a missing file.

THE EXAMINER'S WORDS, UNEDITED

Mention the difference between reading from a file and appending to a file. [2 marks]

## BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

## ANSWER

What is actually asked

One difference, stated. "Mention" signals a short answer, and 2 marks confirms it.

Where it is taught

Module Four, Unit 3.

## MODEL ANSWER

2025/2026 paper, Question Five (c)

Write your answer to **2025/2026 paper, Question Five (c)** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

ANSWER · Q5 (C) READING VS APPENDING, 2 MARKS

**Reading** ("r") takes data **out** of a file and changes nothing. **Appending** ("a") adds new data to the **end**, keeping what is already there. Reading needs the file to exist; appending creates it if it does not. In short: one looks, the other adds without erasing.

## AS PRINTED

2025/2026 paper, Question Five (d)

THE EXAMINER'S WORDS, UNEDITED

The Computer Science Department conducted a Continuous Assessment (CA) for CSC241 students. You have been asked to develop a Python program to manage the students' assessment records. The department requires that all student records be stored in a single text file throughout the program. The file created at the beginning of the program must be used for all subsequent operations. The assessment records should contain the following information for each student: Name, Matric Number, Department, and Score. The records should be stored in a text file named: your\_matric\_number\_CSC241.txt. (Replace your\_matric\_number with your actual matric number. For example: 23CG012345\_CSC241.txt). Write a Python program that uses file handling operations to manage the students' assessment records. A sample record is shown below:

- -Name: John Doe, Matric No: 23CG012345, Department: Computer Science, Score: 78
- -Name: Mary James, Matric No: 23CG012346, Department: MIS, Score: 85

[7.5 marks]

### BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

#### ANSWER

What is actually asked

Four operations on **one** file: write ten records, append five more without destroying them, read all, read the first five. The words "the same file" appear in three of the four requirements.

Where the trap sits

Requirement ii is the whole question. Opening the file the same way you did in i would erase the ten records you just wrote. The mode you choose here is worth more than the rest of the program.

The instruction people miss

The file name must be built from **your own matric number**. Writing the literal example name is a mark lost for not reading.

Where it is taught

Module Four, Unit 3.

### MODEL ANSWER

2025/2026 paper, Question Five (d)

Write your answer to **2025/2026 paper, Question Five (d)** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

#### ANSWER · Q5 (D) THE CA RECORDS FILE, 7.5 MARKS

Write the first records with "w", add more with "a" so the first are kept, then read the whole file and just the first five. The same file name throughout, as the question insists.

ca\_records.py

```
filename = "23CG012345_CSC241.txt"
```

```

# i. first 10 records, "w" starts the file fresh
with open(filename, "w") as f:
    for i in range(10):
        name = input("Name: ")
        matric = input("Matric No: ")
        dept = input("Department: ")
        score = input("Score: ")
        f.write(f"Name: {name}, Matric No: {matric}, "
                f"Department: {dept}, Score: {score}\n")

# ii. 5 more, "a" keeps the first ten
with open(filename, "a") as f:
    for i in range(5):
        name = input("Name: ")
        matric = input("Matric No: ")
        dept = input("Department: ")
        score = input("Score: ")
        f.write(f"Name: {name}, Matric No: {matric}, "
                f"Department: {dept}, Score: {score}\n")

# iii. read and show all
print("All records:")
with open(filename, "r") as f:
    for line in f:
        print(line.strip())

# iv. read and show the first five
print("First five:")
with open(filename, "r") as f:
    for i, line in enumerate(f):
        if i >= 5:
            break
        print(line.strip())

```

The marks live in the modes: "w" once at the start, then "a" so the append does not wipe the ten already written. If both used "w", the second open would erase the first ten before adding five, and you would end with five records, not fifteen. Reading the first five uses enumerate and break to stop early, from Module Three.

## AS PRINTED

2025/2026 paper, Question Six (a)

### THE EXAMINER'S WORDS, UNEDITED

Given the following code segment [3 marks]

```

1
2 nested_list = [
3     [1, 2, 3],

```

```
4      [4, 5, 6],
5      [7, 8, 9],
6      [10, 11, 12, 13],
7      [14, [15, 16, 17], 18],
8      [19, [20, [21, 22], 23], 24]
9  ]
10
```

- i. What is the value at index [3][2]?
- ii. `print (len (nested_list[4]))`
- iii. What is the value at index [5][1][1][0]?

### BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

#### ANSWER

What is actually asked

Three values read out of one nested list. No program: just the answers.

How to read an index chain

Left to right, one bracket at a time, writing down what you are holding after each step. Trying to see [5][1][1][0] in one go is how people lose this mark.

Where the trap sits

The rows are ragged on purpose: row 3 has four items and rows 4 and 5 contain lists inside lists. Counting from 1 instead of 0 gets every one of the three wrong.

Where it is taught

Module Three, Unit 3.

### MODEL ANSWER

2025/2026 paper, Question Six (a)

Write your answer to **2025/2026 paper, Question Six (a)** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

**ANSWER · Q6 (A) NESTED INDEXING, 3 MARKS**

Follow the brackets left to right, one layer at a time. The list is six rows, some of them holding lists of their own.

- `[3][2]`: row 3 is `[10, 11, 12, 13]`, item 2 of it is 12.
- `len(nested_list[4])`: row 4 is `[14, [15, 16, 17], 18]`, which has three items, so 3. The inner list counts as one item.
- `[5][1][1][0]`: row 5 is `[19, [20, [21, 22, 23], 24]]`. `[1]` is `[20, [21, 22, 23], 24]`, its `[1]` is `[21, 22, 23]`, and `[0]` of that is 21.

12

3

21

Lists inside lists, from Module Three, Unit Three. Each bracket peels one layer; write down what each step gives and the chain is harmless.

### AS PRINTED

2025/2026 paper, Question Six (b)

THE EXAMINER'S WORDS, UNEDITED

A fitness club organized a Weekly Step Challenge to encourage members to maintain an active lifestyle. You have been asked to develop a Python program that tracks a participant's daily step count over a specified number of days and determines whether the participant achieved the overall fitness milestone. The club requires that the final performance report be displayed using a void function named `display_achievement_summary()`, which receives the participant's name, total steps taken, and overall target as arguments. Write a Python program that uses a void function to evaluate and display the participant's fitness achievement. [7 marks]

### BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

#### ANSWER

What is actually asked

A program built around **a void function with a named signature**. The question names the function and its three parameters for you, so use those names exactly.

Void is the point

A void function **prints** and returns nothing. Returning the report string instead of printing it answers a different question and loses the marks this part exists to test.

What the marks say

7 marks, and the paper prints a Sample Interaction. Match its wording and layout: it is the marking scheme in disguise.

Where it is taught

Module Four, Unit 1, and Question Three (b) above.

### MODEL ANSWER

2025/2026 paper, Question Six (b)

Write your answer to **2025/2026 paper, Question Six (b)** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

**ANSWER · Q6 (B) THE VOID FUNCTION, 7 MARKS**

A **void** function does the displaying and returns nothing. Read the inputs, sum the daily steps, work out the target, then hand three values to the function.

steps.py

```
def display_achievement_summary(user_name, actual_total, target_total):
    print(f"--- Progress Report for {user_name} ---")
    print("Total Steps:", actual_total)
    if actual_total >= target_total:
        print("Milestone Achieved!!")
    else:
        print("Milestone Not Achieved.")

name = input("Enter your name: ")
days = int(input("Enter the number of days to track: "))
goal = int(input("Enter your daily step goal: "))

total_steps = 0
for day in range(1, days + 1):
    steps = int(input(f"Enter steps for day {day}: "))
    total_steps += steps

target = goal * days
display_achievement_summary(name, total_steps, target)
```

For Alice, 3 days, a 10000 goal, and steps 12000, 9000, 11000:

```
--- Progress Report for Alice ---
Total Steps: 32000
Milestone Achieved!!
```

The target is  $10000 * 3 = 30000$ , and 32000 clears it, so the milestone is achieved. The function **prints and returns nothing**, which is exactly what "void" means: it is called for its effect, not for a value.

### AS PRINTED

2025/2026 paper, Question Six (c)



A commercial renovation company specializes in installing flooring in circular meeting rooms for corporate clients. You have been asked to develop a Python program that estimates the total cost of a flooring project. To promote code reusability and modular programming, the company requires that the program be developed using value-returning functions. The area of the meeting room is calculated using the formula:  $\text{Area} = \pi r^2$  where  $\pi$  should be obtained from the math module. The total project cost is calculated as:  $\text{Total Cost} = (\text{Area} \times \text{Flooring Price}) + \text{Labor Fee}$ . Write a Python program that uses value-returning functions to estimate the cost of the flooring project. [7 marks]

**BREAK IT DOWN FIRST**

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

**ANSWER**

What is actually asked

The mirror of (b): three **value-returning** functions, named for you, that hand results back to a main program which does the printing.

The instruction that carries marks

" $\pi$  should be obtained from the math module" is not a suggestion. Typing 3.14 answers a different question and gives a different number.

Read the sample output twice

The paper prints Total Calculated Cost: 10546.90 for radius 12, price 20, fee 1500. Work it through with `math.pi` before you trust it. The answer below shows what the run actually gives.

Where it is taught

Module Four, Unit 1 and Unit 2 (Modules).

**MODEL ANSWER**

2025/2026 paper, Question Six (c)

Write your answer to **2025/2026 paper, Question Six (c)** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

**ANSWER · Q6 (C) VALUE-RETURNING FUNCTIONS AND MATH.PI, 7 MARKS**

Three value-returning functions, each handing a value back, and `math.pi` for the area of the circle. This is the modular style the question demands.

flooring.py

```
import math
```

```

def get_data():
    radius = float(input("Enter room radius (meters): "))
    price = float(input("Enter flooring price per square meter: "))
    labor = float(input("Enter labor fee: "))
    return radius, price, labor

def calculate_area(radius):
    return math.pi * radius ** 2

def calculate_total(area, material_price, labor_fee):
    return area * material_price + labor_fee

radius, price, labor = get_data()
area = calculate_area(radius)
total_cost = calculate_total(area, price, labor)

print("--- Project Estimate ---")
print(f"Total Calculated Cost: {total_cost:.2f}")
if total_cost > 10000:
    print("Status: Requires Senior Management Approval.")
else:
    print("Status: Approval not required.")

```

For radius 12, price 20, labor 1500:

```

--- Project Estimate ---
Total Calculated Cost: 10547.79
Status: Requires Senior Management Approval.

```

#### TRAP · THE PAPER'S OWN ANSWER IS WRONG

The question sheet prints the sample cost as **10546.90**. Run the arithmetic it asks for and you get **10547.79**. The paper is wrong by 0.89, and it is worth seeing why, because the method is sound and only the printed figure is off.

```

>>> import math
>>> area = math.pi * 12 ** 2
>>> area
452.3893421169302
>>> area * 20 + 1500
10547.786842338604

```

Rounded to two places that is 10547.79. There is no radius, price or fee near the sheet's own sample that reaches 10546.90 with `math.pi`. **Write the working, show the run, and state your figure.** A marker who has run it will credit the correct number; one who has not still sees a fully justified method. This is the same discipline the whole book is built on: trust the run, not the printout, even when the printout is the examiner's.

## Mock Paper One

Mock paper

## Question One

### MODEL ANSWER

### Question One

Write your full answer to **Question One** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

#### ANSWER · QUESTION ONE (A)

Three control structures, one example each

- **Sequence.** Statements run one after another, top to bottom, with no choices. Example: `a = 2` then `b = a + 1`.
- **Selection** (decision). A block runs only if a condition is true. Written with `if`, `elif`, `else`. Example: `if n % 2 == 0: print("even")`.
- **Iteration** (repetition). A block runs many times. Written with `while` or `for`. Example: `for i in range(3): print(i)`.

The question says "give one example of each", so a sentence without code will not score full marks. Write the three lines.

#### ANSWER · QUESTION ONE (B)

The error

The code **does not just print the wrong numbers. It never stops.** It prints 1, then 3, and then hangs forever.

When `n` reaches 5, `continue` jumps straight back to the top of the `while`, **skipping** `n += 1` on the way. So `n` stays 5 for ever and the condition `n <= 9` never turns false.

The fix

```
n = 1
while n <= 9:
    if n % 2 != 0 and n != 5:
        print(n)
    n += 1
```

1  
3  
7  
9

Removing `continue` removes the bug at its root. If you keep `continue`, `n += 1` must happen before it, or the loop hangs again.

**Say the word "infinite" in your answer.** A candidate who writes "it prints 5 by mistake" has not spotted the actual error and will not get the marks.

#### ANSWER · QUESTION ONE (C)

Output

1  
2  
3

### Explanation

The loop counts 1, 2, 3, 4, 5. It prints each until `i` is 4, where `4 % 4 == 0` is true and `break` leaves the loop at once, before the `print`. So 4 and 5 never print.

**"All done" never prints.** A loop's `else` runs only when the loop finishes *normally*. A `break` skips it. Read `else` here as "if no break happened".

This is the marked difference between a good answer and a lucky one. Most candidates add "All done" to the output.

### ANSWER · QUESTION ONE (D)

```
age = int(input("Enter age: "))

if age > 0:
    if age < 13:
        print("Child - N500")
    elif age < 18:
        print("Teen - N800")
    elif age >= 60:
        print("Senior - N600")
    else:
        print("Adult - N1200")
else:
    print("Please enter a valid age")
```

Run with 8, 15, 70, 30 and 0:

```
Child - N500
Teen - N800
Senior - N600
Adult - N1200
Please enter a valid age
```

Why the order is what it is

**The tests must run narrowest first.** `age < 13` before `age < 18`, because a 10 year old satisfies both and the first true test wins. Swap them and no child could ever be priced.

`elif age >= 60` is reached only by people 18 or over, because anything younger was caught above. That is why it needs no upper bound.

### ANSWER · QUESTION ONE (E)

```
line = input("Enter ages separated by spaces: ")
count = 0

for part in line.split():
```

```

age = int(part)
if age > 0:
    if age < 13:
        print(age, "Child - N500")
    elif age < 18:
        print(age, "Teen - N800")
    elif age >= 60:
        print(age, "Senior - N600")
    else:
        print(age, "Adult - N1200")
    count += 1
else:
    print(age, "Please enter a valid age")

print("Tickets priced:", count)

```

Entering 8 15 70 30:

Enter ages separated by spaces: 8 15 70 30  
 8 Child - N500  
 15 Teen - N800  
 70 Senior - N600  
 30 Adult - N1200  
 Tickets priced: 4

What earned the marks

The decision logic is **identical** to part (d). Only two things changed: `split()` replaced the single input, and a for loop wrapped the whole thing. That is what part (e) is testing, every single year. count is incremented inside the positive branch only, because an invalid age is not a ticket.

## Question Two

### MODEL ANSWER

Question Two

Write your full answer to **Question Two** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

ANSWER · QUESTION TWO (A)

| Structure                | Description                            | Real world example                           |
|--------------------------|----------------------------------------|----------------------------------------------|
| <b>List</b> [ ]          | Ordered, changeable, allows duplicates | The register of a class, in order            |
| <b>Tuple</b> ( )         | Ordered, cannot be changed             | The latitude and longitude of a place        |
| <b>Set</b> { }           | Unordered, no duplicates               | The distinct courses a student is taking     |
| <b>Dictionary</b> {k: v} | Pairs looked up by key                 | A student's matric number against their name |

The question asks for a real-world example for each, so a bare definition scores half. Give the example.

## ANSWER · QUESTION TWO (B)

The issues

**The code runs perfectly and prints the wrong answer.** There is no error message to find, which is the difficulty.

Distinct courses: 5

- `list(courses)` copies a list that is already a list. It does nothing useful and removes no duplicates.
- `distinct = courses` merely gives the same list a second name. Nothing was deduplicated at all.

The fix

```
courses = ["CSC241", "MAT121", "CSC241", "PHY121", "MAT121"]
distinct = set(courses)
print("Distinct courses:", len(distinct))
```

Distinct courses: 3

The distinct courses are CSC241, MAT121 and PHY121. The question allows a for loop instead of a set, which is worth the same marks but more writing:

```
distinct = []
for c in courses:
    if c not in distinct:
        distinct.append(c)
```

## ANSWER · QUESTION TWO (C)

Output

```
['CSC241', 'MAT121', 'PHY121']
3
```

Each stage

- `student["courses"]` looks up the key and returns **the list itself**, not a copy. `.append("PHY121")` then changes that list, so the dictionary's value is changed. No reassignment was needed.
- `student["level"] = 300` replaces an existing value. The key already existed, so nothing is added.
- `print(student["courses"])` shows the three courses.
- `len(student)` is **3**, because **len on a dictionary counts its keys**: name, level and courses. It does not count the courses inside it, and it did not change when level was reassigned.

**The trap is the last line.** Answering 4 means you counted something else. There were three keys at the start and three at the end.

## ANSWER · QUESTION TWO (D)

```

quantities = []

for i in range(4):
    q = int(input("Enter quantity: "))
    quantities.append(q)

quantities.sort()
print("Sorted:", quantities)
print("Highest:", max(quantities))
print("Lowest:", min(quantities))

```

Entering 25, 4, 60, 12:

Sorted: [4, 12, 25, 60]

Highest: 60

Lowest: 4

Two ways to lose marks here

- `quantities = quantities.sort()` would leave `None` in the variable and everything after would fail. `sort()` sorts in place and returns nothing. Call it on its own line.
- Forgetting `int()` means you sort strings, so "12" would come before "4" alphabetically.

#### ANSWER · QUESTION TWO (E)

```

line = input("Enter quantities separated by spaces: ")

quantities = []
for part in line.split():
    quantities.append(int(part))

quantities.sort(reverse=True)

reorder = 0
for q in quantities:
    if q < 10:
        reorder += 1

average = sum(quantities) / len(quantities)

print("Sorted:", quantities)
print("Highest:", max(quantities))
print("Lowest:", min(quantities))
print(f"Average: {average:.2f}")
print("Need reordering:", reorder)

```

Entering 25 4 60 12 8:

Enter quantities separated by spaces: 25 4 60 12 8

Sorted: [60, 25, 12, 8, 4]

Highest: 60

Lowest: 4

Average: 21.80  
Need reordering: 2

The details that earn the marks

- `sort(reverse=True)` for descending. The question says descending.
- `{average:.2f}`. The average is a float and will often be a long ugly one. Format it. This is the difference between a professional answer and a puzzling one.
- `q < 10`, strictly below, because the question says "below 10". A quantity of exactly 10 does **not** need reordering. Here 4 and 8 do.

## Question Three

### MODEL ANSWER

### Question Three

Write your full answer to **Question Three** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

#### ANSWER · QUESTION THREE (A)

**What a function is:** a named block of code that runs only when it is called. It may take values in and may hand a value back.

**Why they matter:** reuse (write once, call anywhere); readability (a name says what a block does); maintenance (fix a bug in one place); decomposition (a large problem splits into small testable pieces).

**Built in against user defined:** a built in function comes with Python and is always available (`print`, `len`, `int`, `max`). A user defined function is one you write with `def`, and it must be defined before it is called.

#### The two types of user defined function:

- **Void:** does something and returns nothing. It has no return, so calling it gives `None`. Example: a function that only prints.
- **Value returning:** works something out and hands it back with `return`, so the call can be used as a value.

Four things are asked here and each is worth roughly a quarter. Answer all four explicitly, with a heading for each if it helps you not to forget one.

#### ANSWER · QUESTION THREE (B)

The errors

- **No colon** after `def cube(n)`. This is a `SyntaxError`, found while reading the file, so **nothing runs at all**.
- **Cube with a capital C** does not exist. The function is `cube`, and names are case sensitive.

Only the first is reported. Fix it, run again, and only then do you see:



NameError: name 'Cube' is not defined. Did you mean: 'cube'?

The fix

```
def cube(n):  
    return n ** 3  
  
print(cube(3))
```

27

Say **both** errors. The question says "errors", plural, and the marks are split. Say also that the first hides the second: it shows you understand why a SyntaxError stops everything.

ANSWER · QUESTION THREE (C)

Output

```
110.0  
120.0
```

Explanation

- `tax=0.1` is a **default parameter**: a value used when the caller does not supply one.
- `total(100)` passes only price, so tax falls back to `0.1`. The result is  $100 + 10.0 = \mathbf{110.0}$ .
- `total(100, 0.2)` supplies tax, and the argument **overrides** the default. The result is  $100 + 20.0 = \mathbf{120.0}$ .

**Both answers are floats**, printed as `110.0` and `120.0`, not `110` and `120`. The multiplication by a float makes the whole expression a float. Writing `110` without the decimal point is not the output.

Worth stating in your answer: a parameter with a default must come after any without one, so `def total(tax=0.1, price)` would be a SyntaxError.

ANSWER · QUESTION THREE (D)

```
def grade(score):  
    if score >= 70:  
        return "A"  
    elif score >= 60:  
        return "B"  
    elif score >= 50:  
        return "C"  
    else:  
        return "F"  
  
for i in range(3):  
    name = input("Student name: ")  
    score = int(input("Score: "))  
    print(f"Student: {name} | Score: {score} | Grade: {grade(score)}")
```

Entering Ada 72, Bola 45, John 65:

Student: Ada | Score: 72 | Grade: A  
Student: Bola | Score: 45 | Grade: F  
Student: John | Score: 65 | Grade: B

Two things the marker is looking for

- **return, not print.** The question says the function "returns" the letter. A function that prints instead hands back None, and the caller has nothing to display.
- **Descending order of tests.** 70 first. Test 50 first and every score above 50 would be a C.

grade(score) is called **inside the f-string**, which is neat and legal. Assigning it to a variable first is equally correct.

#### ANSWER · QUESTION THREE (E)

```
def grade(score):
    if score >= 70:
        return "A"
    elif score >= 60:
        return "B"
    elif score >= 50:
        return "C"
    else:
        return "F"

total = 0
passed = 0

while True:
    name = input("Student name (or 'done'): ")
    if name == "done":
        break
    score = int(input("Score: "))
    print(f"Student: {name} | Score: {score} | Grade: {grade(score)}")
    total += 1
    if score >= 50:
        passed += 1

print(f"Students: {total}, Passed: {passed}, Failed: {total - passed}")
```

Entering Ada 72, Bola 45, John 65, then done:

Student: Ada | Score: 72 | Grade: A  
Student: Bola | Score: 45 | Grade: F  
Student: John | Score: 65 | Grade: B  
Students: 3, Passed: 2, Failed: 1

The shape

**The function is untouched.** Only the loop changed, from `for i in range(3)` to a sentinel `while True` with a `break`. That is the whole of part (e), and it is why a function is worth writing in part (d) in the first place.

**Test for 'done' before asking for the score**, or the program will demand a score for a student who does not exist.

A pass is 50 and above, so the test is  $\geq$ . Bola's 45 fails.

## Question Four

### MODEL ANSWER

### Question Four

Write your full answer to **Question Four** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

#### ANSWER · QUESTION FOUR (A)

**What a string is:** a sequence of characters, written in single or double quotes. It is **immutable**, so it cannot be changed in place.

| Method                     | Explanation                                           | Example                              | Gives              |
|----------------------------|-------------------------------------------------------|--------------------------------------|--------------------|
| <code>upper()</code>       | Returns a copy in capitals                            | <code>"ada".upper()</code>           | <code>'ADA'</code> |
| <code>lower()</code>       | Returns a copy in small letters                       | <code>"ADA".lower()</code>           | <code>'ada'</code> |
| <code>strip()</code>       | Returns a copy with whitespace removed from both ends | <code>" hi ".strip()</code>          | <code>'hi'</code>  |
| <code>replace(a, b)</code> | Returns a copy with every a changed to b              | <code>"a-b".replace("-", " ")</code> | <code>'a b'</code> |

Notice every explanation says **"returns a copy"**. That single word shows the marker you understand immutability, and it costs you nothing to write.

#### ANSWER · QUESTION FOUR (B)

The errors

- `full_name.index()` is called with **nothing to search for**. It needs the text whose position you want, here the space. `TypeError: index expected at least 1 argument, got 0`
- `lowercase()` **does not exist**. The method is `lower()`.

The subtlety that costs half the marks

Fixing `index()` to `index(" ")` is not enough. That gives the position **of the space**, which is 5, and slicing from 5 **includes the space**:

```
>>> full_name = "Grace Hopper"
>>> full_name[full_name.index(" "):]
' Hopper'
```

Note the leading space inside the quotes. You must start **one past** it:

The fix

```
full_name = "Grace Hopper"
surname = full_name[full_name.index(" ") + 1:]
```

```
print("Surname:", surname.lower())
```

Surname: hopper

A cleaner answer, worth full marks and less error prone:

```
surname = full_name.split()[1]
print("Surname:", surname.lower())
```

#### ANSWER · QUESTION FOUR (C)

Output

```
CSC241 Python
    CSC241 CODE
13
```

Each line

- `tag.strip()` removes the three spaces at each end and returns CSC241 Python. **tag itself is unchanged.**
- `tag.upper()` gives " CSC241 PYTHON ". **Now** `replace("PYTHON", "CODE")` matches, because the text is in capitals by the time `replace` sees it. **The spaces are still there**, because `strip` on the line before returned a new string and changed nothing.
- `len(tag.strip())` is **13**: six for CSC241, one space, six for Python. Note it is `len` of the *stripped* string, not of `tag`, which is 19.

**Two traps in three lines.** The middle line only matches because of the `upper()` before it, and the last line only gives 13 because of the `strip()` inside it.

#### ANSWER · QUESTION FOUR (D)

```
regs = [
    "24/CSC/001",
    "23/CSC/117",
    "24/MAT/042",
    "22/PHY/008",
    "24/CSC/236",
]

for reg in regs:
    year = reg.split("/")[0]
    print(f"Reg: {reg} | Year: {year}")
```

```
Reg: 24/CSC/001 | Year: 24
Reg: 23/CSC/117 | Year: 23
Reg: 24/MAT/042 | Year: 24
Reg: 22/PHY/008 | Year: 22
Reg: 24/CSC/236 | Year: 24
```

Why index 0

"24/CSC/001".split("/") gives ['24', 'CSC', '001']. The year is the part before the first slash, so it is item **0**. The separator itself is thrown away.

The year stays a **string**, which is correct here: it is a label, not a number, and "001" would lose its zeros as an integer.

#### ANSWER · QUESTION FOUR (E)

```
paragraph = input("Enter a paragraph: ")

for sentence in paragraph.split("."):
    sentence = sentence.strip()
    if sentence == "":
        continue

    words = len(sentence.split())

    consonants = 0
    for ch in sentence.lower():
        if ch.isalpha() and ch not in "aeiou":
            consonants += 1

    print(f"Sentence: {sentence.lower()}")
    print(f" Words: {words}, Consonants: {consonants}")
```

Entering Python is fun. I like code.

Enter a paragraph: Python is fun. I like code.

Sentence: python is fun

Words: 3, Consonants: 8

Sentence: i like code

Words: 3, Consonants: 4

The one that catches everybody

**ch.isalpha() is not optional.** A consonant is a *letter* that is not a vowel. Without that test the **spaces** get counted, since a space is certainly not in "aeiou", and the answer comes out as 10 instead of 8.

The eight consonants in "python is fun" are p, y, t, h and n from *python*, s from *is*, then f and n from *fun*. The vowels o, i and u are excluded, and the two spaces are excluded by `isalpha()`.

**y counts as a consonant here**, because the question defines vowels as a, e, i, o, u and y is not among them. Follow the question's definition, not your instinct about English.

The `continue` is needed because the paragraph ends in a full stop, so splitting leaves an empty final piece.

## Question Five

Write your full answer to **Question Five** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

#### ANSWER · QUESTION FIVE (A)

**File handling** is working with files on disk from a program, so that data survives after the program ends. It is always three steps: open, process, close.

| Mode | Name             | Behaviour                                                              |
|------|------------------|------------------------------------------------------------------------|
| "r"  | Read             | Opens for reading. <b>Error if the file does not exist.</b>            |
| "w"  | Write            | <b>Erases the file completely</b> , then writes. Creates it if absent. |
| "a"  | Append           | Adds to the end, keeping what is there. Creates it if absent.          |
| "x"  | Exclusive create | Creates it. <b>Error if it already exists.</b>                         |

**Reading against appending:** reading takes data *out* of the file and changes nothing, and the file must already exist. Appending puts data *into* the file at the end, keeps everything already there, and creates the file if needed.

Worth adding, and it shows real understanding: the dangerous distinction is "w" against "a". Opening with "w" destroys the contents immediately, before you write anything.

#### ANSWER · QUESTION FIVE (B)

The errors

- The write line is **missing its closing bracket**. This is a `SyntaxError`, so **nothing in the file runs**.
- `f.close` has **no brackets**, so it is not a call. It names the method and discards it. The file is never closed.

**The first hides the second**, and the second is the more dangerous, because it reports nothing at all: no error, no message, and possibly an empty file.

The fix

```
f = open("report.txt", "w")
f.write("Weekly Report")
f.close()
```

Better still, and worth saying:

```
with open("report.txt", "w") as f:
    f.write("Weekly Report")
```

`with` closes the file for you, so the second bug becomes impossible.

#### ANSWER · QUESTION FIVE (C)

Result

Log updated.

Each line

- `with open("log.txt", "a") as f` opens `log.txt` in **append** mode, **creating it if it does not exist**, and calls it `f`.
- `f.write("Entry recorded.\n")` adds that line **to the end** of the file, keeping everything already in it. The `\n` is needed because `write` adds no newline of its own.
- The block ends, so with **closes the file automatically**. There is no `close()` to forget.
- `print("Log updated.")` displays that text on screen.

**Only "Log updated." appears on screen.** The written line went into the file, not to the display. Saying so is the mark. Run it three times and the file has three entries, while the screen still shows one line each time.

#### ANSWER · QUESTION FIVE (D)

```
count = int(input("How many books were borrowed? "))

with open("borrowed.txt", "w") as f:
    for i in range(count):
        title = input("Title: ")
        f.write(title + "\n")

print("Log saved successfully.")
```

Entering 2, then Things Fall Apart, then Purple Hibiscus:

Log saved successfully.

and `borrowed.txt` now holds:

Things Fall Apart  
Purple Hibiscus

The detail

+ **"\n" is essential.** The requirement says "on a new line", and `write` adds none. Without it the file would hold Things Fall ApartPurple Hibiscus on one line. An f-string works equally well:  
`f.write(f"{title}\n")`.

#### ANSWER · QUESTION FIVE (E)

```
with open("fines.txt", "w") as f:
    while True:
        title = input("Title (or 'done'): ")
        if title == "done":
            break
        days = int(input("Days overdue: "))
        f.write(f"{title}, {days}\n")

total_fine = 0
with open("fines.txt", "r") as f:
```

```

for line in f:
    title, days = line.strip().split(", ")
    days = int(days)
    fine = days * 50
    total_fine += fine
    print(f"{title}: {days} days, N{fine}")

print("Total fine: N" + str(total_fine))

```

Entering Things Fall Apart 3, Purple Hibiscus 0, Half of a Yellow Sun 2, then done:

```

Things Fall Apart: 3 days, N150
Purple Hibiscus: 0 days, N0
Half of a Yellow Sun: 2 days, N100
Total fine: N250

```

The details that decide this

- **line.strip() before split(", ")**. Every line read from a file keeps its `\n`. Without the strip, days would be `"3\n"`, and `int("3\n")` happens to work, but the habit will bite you elsewhere. Strip first, always.
- **split(", ") with the space**, matching exactly the format you wrote. Splitting on `","` alone leaves a leading space on the number.
- The fine is N50 per day, so a book 0 days overdue owes N0 and still appears in the list. Do not skip it.

## Question Six

### MODEL ANSWER

Question Six

Write your full answer to **Question Six** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

#### ANSWER · QUESTION SIX (A)

**i. What a database is, and why:** an organised collection of data held in tables of rows and columns, stored so it can be searched, changed and retrieved efficiently. It is used because the data lasts beyond the program; it can be searched without reading everything; it enforces structure, so a column keeps its type; it serves many users at once; and it keeps data consistent through rules such as a primary key.

**ii. The role of sqlite3:** it is the built in Python module that connects a Python program to an SQLite database file, letting you create tables, insert rows and query them from Python. It ships with Python, so nothing needs installing.

**iii. Three SQL commands:**

| Command | Function |
|---------|----------|
|---------|----------|



|              |                                           |
|--------------|-------------------------------------------|
| CREATE TABLE | Makes a new table and defines its columns |
| INSERT INTO  | Adds a new row of data to a table         |
| SELECT       | Retrieves rows from a table               |

#### ANSWER · QUESTION SIX (B)

The issues

**Both are missing brackets**, and they behave very differently.

- `conn.cursor` does not call the method, so cursor holds the method itself rather than a cursor. The next line then fails with `AttributeError: 'builtin_function_or_method' object has no attribute 'execute'`.
- `conn.commit` does nothing at all, **silently**. The table would never be saved.

The fix

```
import sqlite3
conn = sqlite3.connect("library.db")
cursor = conn.cursor()
cursor.execute("CREATE TABLE IF NOT EXISTS books (id INTEGER PRIMARY KEY, title TEXT)")
conn.commit()
conn.close()
```

**Worth the extra mark:** add `IF NOT EXISTS`. Without it the fixed program works once and then fails on every later run with `sqlite3.OperationalError: table books already exists`, because connect keeps the file between runs.

#### ANSWER · QUESTION SIX (C)

**i. What it does:** connects to `library.db`, creating the file if it does not exist; gets a cursor; runs an `INSERT` adding one row to the `books` table with the title 'Things Fall Apart', letting `id` fill itself; commits, making the change permanent; and closes the connection.

**ii. What happens when it runs: nothing is displayed at all.** There is no `print` and no `SELECT`, so a successful run produces no output. One row is silently added to the table.

**iii. The assumptions:**

- The `books` table **must already exist**, with a `title` column. This code never creates it, so without it you get `sqlite3.OperationalError: no such table: books`.
- `id` must fill itself, that is, be declared `INTEGER PRIMARY KEY`, since no `id` is supplied.
- The file must be writable, and the disk must have room.

Part ii is the easy mark most candidates miss. "No output" is the answer, and it is worth saying in those words.

#### ANSWER · QUESTION SIX (D)

```
import sqlite3
```

```

conn = sqlite3.connect("registry.db")
cursor = conn.cursor()

cursor.execute("""CREATE TABLE IF NOT EXISTS students
                  (id INTEGER PRIMARY KEY, name TEXT, matric TEXT)""")

for i in range(3):
    name = input("Name: ")
    matric = input("Matric: ")
    cursor.execute("INSERT INTO students (name, matric) VALUES (?, ?)", (name, matric))

conn.commit()
conn.close()
print("Students saved.")

```

Entering Ada 24/CSC/001, Bola 24/CSC/117, John 24/MAT/042:

Students saved.

What earns the marks

- **IF NOT EXISTS**, because the question says "if it does not already exist" in as many words.
- **The ? placeholders**, not string joining. A matric number contains slashes and a name may contain an apostrophe, which would break joined SQL outright.
- **matric is TEXT**, not INTEGER. It contains slashes, so it is not a number at all.
- `commit()` before `close()`, or nothing is saved.

#### ANSWER · QUESTION SIX (E)

```

import sqlite3

conn = sqlite3.connect("registry.db")
cursor = conn.cursor()
cursor.execute("""CREATE TABLE IF NOT EXISTS students
                  (id INTEGER PRIMARY KEY, name TEXT, matric TEXT)""")

while True:
    name = input("Name (or 'done'): ")
    if name == "done":
        break
    matric = input("Matric: ")
    cursor.execute("INSERT INTO students (name, matric) VALUES (?, ?)", (name, matric))

conn.commit()

cursor.execute("SELECT * FROM students")
for row in cursor.fetchall():
    print(f"{row[0]} | {row[1]} | {row[2]}")

conn.close()

```

Entering Ada 24/CSC/001, Bola 24/CSC/117, then done:

```
1 | Ada | 24/CSC/001
2 | Bola | 24/CSC/117
```

The shape, one last time

The sentinel loop is **exactly** the one from Questions Three and Five. The only new thing in this question is the `SELECT` and `fetchall()` at the end, which the question asks for in its last line.

**Each row is a tuple**, so `row[0]`, `row[1]` and `row[2]` are id, name and matric, in the order the table declares them.

**Commit before you select.**

## After you have marked it

### Mock Paper Two

Mock paper

### Question One

#### MODEL ANSWER

Question One

Write your full answer to **Question One** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

#### ANSWER · Q1 (A)

A **control structure** decides which statements run, and how often. Without one a program is a list that runs top to bottom, once.

- **break** leaves the loop immediately. Nothing after it in the loop body runs, and the loop does not continue. `for i in range(5): if i == 2: break`
- **continue** abandons *this pass only* and jumps to the next. The loop carries on. `for i in range(5): if i % 2: continue`
- **else on a loop** runs when the loop finishes **without being broken**. It is the part everyone forgets: it is not "otherwise", it is "and we got all the way through".

The three-way answer the examiner wants: sequence, selection (`if`), repetition (`for`, `while`). `break` and `continue` steer the third.

#### ANSWER · Q1 (B)

**Two errors, and the second only appears once the first is gone.**

1. The `for` line has **no colon**. Python stops before running anything: `SyntaxError: expected ':'`
2. Add the colon and it runs, and now fails on the last line: `NameError: name 'Total' is not defined`  
`Total` is not `total`. Python is case sensitive, and a name with a capital letter is a different name.

fixed

```
total = 0
for i in range(1, 5):
    total += i
print("Total:", total)
```

Total: 10

`range(1, 5)` gives 1, 2, 3, 4. The 5 is the stop and never arrives, so the total is 10 and not 15. If you answered 15 you have the off by one that this part exists to catch.

ANSWER · Q1 (C)

```
3
2
1
Liftoff
```

`range(3, 0, -1)` counts **down**: start at 3, stop before 0, step by -1. So 3, 2, 1, and never 0.

**Why the last line prints at all** is the whole question. A loop's `else` runs when the loop **completes without a break**. There is no break here, so the loop finishes, and the `else` fires.

This is the mirror of Mock One Q1(c), where a `break` did fire and the `else` was skipped. Same rule, both sides of it. If you can state that rule in one sentence you have both marks.

ANSWER · Q1 (D)

bill.py

```
units = int(input("Enter units consumed: "))

if units > 0:
    if units < 50:
        band, rate = "Lifeline", 10
    elif units < 200:
        band, rate = "Domestic", 20
    else:
        band, rate = "Commercial", 30
    print(f"Band: {band} | Bill: N{units * rate}")
else:
    print("Please enter a valid reading")
```

Entering 120:

Band: Domestic | Bill: N2400

The `elif` chain is doing work you get for free: by the time Python reaches `elif units < 200` it already knows the units are **not** under 50, so you never write `units >= 50` and `units < 200`. Writing that instead is not wrong, but it is the long way round and it is where people put the boundary in the wrong place.

### The boundaries decide the mark:

| Units | Band       | Bill  |
|-------|------------|-------|
| 49    | Lifeline   | N490  |
| 50    | Domestic   | N1000 |
| 199   | Domestic   | N3980 |
| 200   | Commercial | N6000 |

50 is **not** under 50, so it is Domestic. 200 is not under 200, so it is Commercial. Read < as "strictly less than" every time, and these four fall out on their own.

#### ANSWER · Q1 (E)

Same banding, wrapped in **split and convert**. Nothing about the pricing changes. That is the point of "as an extension of (d)": do not rewrite it, wrap it.

bills.py

```
line = input("Enter the readings separated by spaces: ")

count = 0
grand = 0
for part in line.split():
    units = int(part)
    if units > 0:
        if units < 50:
            band, rate = "Lifeline", 10
        elif units < 200:
            band, rate = "Domestic", 20
        else:
            band, rate = "Commercial", 30
        bill = units * rate
        grand += bill
        count += 1
        print(f"{units} units | {band} | N{bill}")
    else:
        print(units, "Please enter a valid reading")

print("Readings billed:", count)
print("Total:", grand)
```

Entering 30 120 250:

```
30 units | Lifeline | N300
120 units | Domestic | N2400
250 units | Commercial | N7500
Readings billed: 3
Total: 10200
```

Three things earn the marks: `split()` with no argument (any run of spaces), `int(part)` on every piece because `input()` hands you text, and count incremented **inside** the valid branch, so a

rejected reading is not counted as billed.

## Question Two

### MODEL ANSWER

Question Two

Write your full answer to **Question Two** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

ANSWER · Q2 (A)

| Structure                   | What it is                              | Use it for                                                         |
|-----------------------------|-----------------------------------------|--------------------------------------------------------------------|
| <b>List</b> [ ]             | Ordered, changeable, duplicates allowed | The parcels weighed today: the set of them changes all day.        |
| <b>Tuple</b> ( )            | Ordered, <b>unchangeable</b>            | A fixed pair like (latitude, longitude), or the days of the week.  |
| <b>Set</b> { }              | Unordered, <b>no duplicates</b>         | The distinct courses a student takes: ask it "how many different?" |
| <b>Dictionary</b><br>{k: v} | Keyed pairs                             | Stock levels: look up "pen" directly, without searching.           |

**List against tuple:** use a tuple when the collection must not change, and the fact that it cannot is the protection. A list is the default; reach for a tuple deliberately.

**What a dictionary gives that a list cannot:** it finds a value **by name**. In a list you must know the position, or search the whole thing. In a dictionary `stock["pen"]` goes straight there, however many items there are.

ANSWER · Q2 (B)

**Two errors, the second hidden behind the first.**

- `weights` is a **tuple**, and a tuple cannot be changed: `AttributeError: 'tuple' object has no attribute 'append'` The collection is being added to, so it should have been a list: `[12, 5, 20, 8]`.
- Make it a list and the crash stops, but the output is wrong: `Sorted: None` `sort()` sorts the list **in place** and returns `None`. You printed its return value, which is nothing. Sort first, then print the list.  
fixed

```
weights = [12, 5, 20, 8]
weights.append(15)
weights.sort()
print("Sorted:", weights)
```

Sorted: [5, 8, 12, 15, 20]

The second error is the one worth remembering: **a method that changes a thing in place gives back None**. `sort()`, `reverse()` and `append()` are all like this. If you want a sorted copy and the

original kept, that is `sorted(weights)`, which returns a new list.

**ANSWER · Q2 (C)**

9  
3  
True

1. `stock["pen"] = stock["pen"] - 3` reads 12, takes 3, and writes 9 back to the **existing** key. The dictionary still has two keys.
2. `stock["bag"] = 7` uses a key that is not there, so it is **created**. The same square brackets both update and add: which one happens depends entirely on whether the key already exists.
3. `print(stock["pen"])` gives **9**.
4. `len(stock)` counts the **keys**: pen, book, bag, so **3**. Not the values, and not the characters.
5. `"book" in stock` asks whether that is a **key**, and it is: **True**. On a dictionary, `in` looks at the keys, never the values. `12 in stock` would be False even though 12 is in there.

**ANSWER · Q2 (D)**

parcels.py

```
weights = []

for i in range(4):
    w = int(input(f"Weight of parcel {i + 1}: "))
    weights.append(w)

weights.sort()
print("Sorted:", weights)
print("Heaviest:", max(weights))
print("Lightest:", min(weights))
```

Entering 12, 5, 20, 8:

Sorted: [5, 8, 12, 20]

Heaviest: 20

Lightest: 5

"Appropriate built-in functions" means `max()` and `min()`. You could take `weights[-1]` and `weights[0]` after sorting and be right, but only because you sorted first; `max()` needs no such promise and says what it means.

**ANSWER · Q2 (E)**

parcels\_any.py

```
line = input("Enter all the weights separated by spaces: ")

weights = []
for part in line.split():
    weights.append(int(part))
```

```

weights.sort(reverse=True)
average = sum(weights) / len(weights)

surcharge = 0
for w in weights:
    if w > 20:
        surcharge += 1

print("Sorted:", weights)
print("Heaviest:", max(weights))
print("Lightest:", min(weights))
print(f"Average: {average:.2f}")
print("Surcharged:", surcharge)

```

Entering 12 5 20 8 25:

```

Sorted: [25, 20, 12, 8, 5]
Heaviest: 25
Lightest: 5
Average: 14.00
Surcharged: 1

```

- **Descending** is `sort(reverse=True)`. It still returns `None`, so it is still a statement on its own line.
- **Only one parcel is surcharged.** The rule is *above* 20, so the 20 kg parcel does not qualify: 25 does. If you answered 2 you read `>` as `>=`, which is the single most expensive habit in this paper.
- The average is 70 divided by 5, which is 14.0 exactly. `{average:.2f}` shows it as 14.00: two decimal places for money and measurements, always.

## Question Three

### MODEL ANSWER

### Question Three

Write your full answer to **Question Three** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

ANSWER · Q3 (A)

- **Parameter against argument.** The **parameter** is the name in the definition, `def fee(hours)`. The **argument** is the actual value you pass, `fee(3)`. One is the label, the other is the thing.
- **return** hands a value back to whoever called the function, and ends the function there and then. Leave it out and the function still returns something: it returns **None**. That is not an error, which is exactly why it is dangerous, and it is what part (b) is about.
- **A default parameter** supplies a value for an argument the caller does not give: `def pay(hours, rate=500)`. It **must come last** because Python matches arguments to parameters by position. If a default came first, then `pay(8)` could not say whether the 8 is the defaulted one or the required one. Python refuses the definition outright rather than guess.

ANSWER · Q3 (B)



## Two errors, and the second is invisible until the first is fixed.

1. `def fee(hours)` has **no colon**. Nothing runs at all: `SyntaxError: expected ':'`
2. Add it and the program runs cleanly and prints the wrong thing: `None` The function works out total and then **throws it away**. There is no return, so the function gives back `None`, and print faithfully prints it. No error, no warning, no marks.

fixed

```
def fee(hours):  
    return 200 + (hours - 1) * 150  
  
print(fee(3))
```

500

200 for the first hour, then 2 more hours at 150, so  $200 + 300 = 500$ . **A function that computes and does not return is the most common quiet failure in this course.** If your function prints `None`, this is why, every time.

ANSWER · Q3 (C)

4000

5600

- `pay(8)` gives one argument, so `hours` is 8 and `rate` falls back to its default of 500:  $8 * 500 = 4000$ .
- `pay(8, 700)` gives two, so the second **replaces** the default:  $8 * 700 = 5600$ .

The default is not a constant. It is the value used *only* when the caller stays silent. That is what makes `rate` optional without making it fixed, and it is the whole reason default parameters exist.

ANSWER · Q3 (D)

carpark.py

```
def fee(hours):  
    return 200 + (hours - 1) * 150  
  
for i in range(3):  
    plate = input("Plate number: ")  
    hours = int(input("Hours parked: "))  
    print(f"Plate: {plate} | Hours: {hours} | Fee: N{fee(hours)}")
```

Entering ABC123/3, XYZ789/1 and KJA55/5:

Plate: ABC123 | Hours: 3 | Fee: N500

Plate: XYZ789 | Hours: 1 | Fee: N200

Plate: KJA55 | Hours: 5 | Fee: N800

Check the one-hour case by hand, because it is the one the formula can get wrong:  $200 + (1 - 1) * 150 = 200 + 0 = 200$ . Correct. Write the formula as  $200 + \text{hours} * 150$  and that vehicle is charged

350 for its first hour, and every single answer is 150 too high.

#### ANSWER · Q3 (E)

The **sentinel loop**: while True, ask, and break on the stop word. The function itself does not change at all.

carpark\_any.py

```
def fee(hours):
    return 200 + (hours - 1) * 150

count = 0
takings = 0

while True:
    plate = input("Plate number (or 'done'): ")
    if plate == "done":
        break
    hours = int(input("Hours parked: "))
    amount = fee(hours)
    takings += amount
    count += 1
    print(f"Plate: {plate} | Hours: {hours} | Fee: N{amount}")

print("Vehicles charged:", count)
print(f"Total takings: N{takings}")
```

Entering ABC123/3, XYZ789/1, then done:

```
Plate: ABC123 | Hours: 3 | Fee: N500
Plate: XYZ789 | Hours: 1 | Fee: N200
Vehicles charged: 2
Total takings: N700
```

**Test the stop word before you ask for the hours.** Put the `int(input(...))` above the check and typing done makes the program demand the hours for a vehicle that does not exist, and then crash on `int("")`. The check goes immediately after the input it checks.

## Question Four

### MODEL ANSWER

Question Four

Write your full answer to **Question Four** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

#### ANSWER · Q4 (A)

A **string** is text: a sequence of characters in quotes. It is **immutable**, meaning it can never be changed after it is made.

**What follows from that** is the examinable half: a string method can never change the string it is called on, so it hands you a **new** one. `name.upper()` leaves `name` exactly as it was. If you do not catch the result, with `name = name.upper()`, the work is thrown away and nothing happens. This is the opposite of a list's `sort()`, which changes the list and hands back nothing.

| Method                  | Does                                    | Example                       | Gives                   |
|-------------------------|-----------------------------------------|-------------------------------|-------------------------|
| <code>upper()</code>    | Returns a capitalised copy              | <code>"ada".upper()</code>    | <code>'ADA'</code>      |
| <code>strip()</code>    | Returns a copy with the ends trimmed    | <code>" hi ".strip()</code>   | <code>'hi'</code>       |
| <code>split(sep)</code> | Cuts it into a list                     | <code>"a,b".split(",")</code> | <code>['a', 'b']</code> |
| <code>isalpha()</code>  | <b>Asks</b> a question, changes nothing | <code>"Ada".isalpha()</code>  | <code>True</code>       |

The fourth is the one the question is fishing for. `isalpha()`, `startswith()` and `find()` interrogate a string rather than transform it; the rest hand back a changed copy.

ANSWER · Q4 (B)

**Two errors, the second wearing the first as a disguise.**

- `split("@")` gives a **list**, `['Ada', 'CU.edu.ng']`, not a string. So `domain` is a list, and a list has no `lower`: `AttributeError: 'list' object has no attribute 'lower'` You want the piece after the `@`, which is index **1**: `email.split("@")[1]`.
- Take the index and the crash goes away, and the output is still wrong: `Domain: <built-in method lower of str object at 0x...>` `domain.lower` has **no brackets**, so it never calls the method. It names it. Python prints the method itself, which is a real object and prints happily. (The `0x...` is a memory address, and it is a different number every time the program runs.)

fixed

```
email = "Ada@CU.edu.ng"
domain = email.split("@")[1]
print("Domain:", domain.lower())
```

Domain: cu.edu.ng

**The missing brackets again.** This is the third time this manual has shown you the same fault: `file.close`, `conn.cursor`, and now `domain.lower`. **A method without brackets is not called, and Python never complains.** When output looks like `<built-in method ...>` or `<bound method ...>`, you have found it.

ANSWER · Q4 (C)

```
CSC241-PYTHON
['csc241', 'python']
13
```

- `code.strip()` gives `'csc241-python'`, and `.upper()` capitalises that copy. The chain reads left to right: strip first, then capitalise the result.

2. `split("-")` cuts the stripped string at the hyphen into **two** pieces. They stay lowercase, because `upper()` on the line above returned a new string and did not touch code.
3. `len(code.strip())` is **13**: c-s-c-2-4-1 is 6, the hyphen is 1, p-y-t-h-o-n is 6. `len(code)` without the strip would be **17**, counting the four spaces.

code itself is unchanged throughout, spaces and all. Three methods were called on it and not one of them altered it: that is immutability doing exactly what part (a) says it does.

#### ANSWER · Q4 (D)

domains.py

```
emails = ["ada@cu.edu.ng", "bola@gmail.com", "chidi@yahoo.com",
          "dele@cu.edu.ng", "efe@outlook.com"]

for email in emails:
    domain = email.split("@")[1]
    print(f"Email: {email} | Domain: {domain}")
```

```
Email: ada@cu.edu.ng | Domain: cu.edu.ng
Email: bola@gmail.com | Domain: gmail.com
Email: chidi@yahoo.com | Domain: yahoo.com
Email: dele@cu.edu.ng | Domain: cu.edu.ng
Email: efe@outlook.com | Domain: outlook.com
```

Index **1**, not 0. Splitting "ada@cu.edu.ng" at the @ gives ['ada', 'cu.edu.ng']: the name is 0 and the domain is 1. The dots in cu.edu.ng are harmless, because we split on @ and the dots are all inside a piece we never cut again.

#### ANSWER · Q4 (E)

codes.py

```
line = input("Enter the codes separated by commas: ")

for code in line.split(","):
    code = code.strip()
    if code == "":
        continue
    code = code.upper()

    letters = 0
    for ch in code:
        if ch.isalpha():
            letters += 1

    others = len(code) - letters
    csc = code.startswith("CSC")
    print(f"Code: {code} | Letters: {letters} | Not letters: {others} | CSC: {csc}")
```

Entering csc241, mat121x, , abc:

Code: CSC241 | Letters: 3 | Not letters: 3 | CSC: True  
Code: MAT121X | Letters: 4 | Not letters: 3 | CSC: False  
Code: ABC | Letters: 3 | Not letters: 0 | CSC: False

- **Four pieces went in and three came out.** The empty one between the two commas is skipped by the `continue`, exactly as the requirement asks. This is the same guard as Mock One's paragraph, and it exists because real input has gaps in it.
- **Capitalise before you test.** `startswith("CSC")` is case sensitive, so `"csc241"` would answer False if you tested it before `upper()`. Order matters, and the requirement lists it in the right order for a reason.
- **Not letters is `len(code) - letters`,** not a second loop. You have already counted the letters, and everything else is what remains.

## Question Five

### MODEL ANSWER

Question Five

Write your full answer to **Question Five** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

ANSWER · Q5 (A)

**with open(...)** closes the file for you, whatever happens, even if the code inside raises an error. A plain `open()` leaves that to you, and if an exception fires before your `close()`, the file is never closed and the last thing you wrote may never reach the disk. `with` is not a convenience; it is the guarantee.

| Mode | Does                                        | If the file does not exist             |
|------|---------------------------------------------|----------------------------------------|
| "r"  | Read only. Cannot write.                    | <b>Error.</b> It refuses to create it. |
| "w"  | Write, and <b>empty the file first</b> .    | Creates it.                            |
| "a"  | Append: add to the end, keep what is there. | Creates it.                            |

**Reading against appending:** "r" takes data out and changes nothing; "a" puts data in at the end and destroys nothing. The dangerous one is "w", which silently throws away the whole file the instant you open it.

ANSWER · Q5 (B)

### Two errors.

1. "r" is **read only**, and the program is trying to write. The mode should be "a" to add a name and keep the others.
2. `f.close` has **no brackets**, so the file is never closed. Same fault as Q4(b), in a different coat.

**The first error gives two different messages, and which one you get depends on something the code does not show you:**

| If names.txt   | Python says                                                         |
|----------------|---------------------------------------------------------------------|
| exists         | UnsupportedOperation: not writable                                  |
| does not exist | FileNotFoundError: [Errno 2] No such file or directory: 'names.txt' |

With the file missing, `open()` itself fails and `write` is never reached, so you never hear about writability at all. Both answers are correct, and saying **why there are two** is worth more than either.

fixed

```
f = open("names.txt", "a")
f.write("Ada\n")
f.close()
```

Better still, let `with` do the closing and the second error cannot happen at all:

```
with open("names.txt", "a") as f:
    f.write("Ada\n")
```

#### ANSWER · Q5 (C)

second

Followed by a blank line, because the file's contents end in `\n` and `print` adds another.

**The first line is gone.** Opening in `"w"` empties the file immediately, before a single byte is written. The second `with` block therefore starts from nothing, and `"first"` is destroyed without a word.

Had the second block used `"a"`, the file would hold both lines. This one question is the whole difference between the two modes, and it is why a program that saves records in a loop must open the file **once**, outside the loop: reopen it in `"w"` each time round and you finish with a file containing only the last record.

#### ANSWER · Q5 (D)

attendance.py

```
count = int(input("How many students are present? "))

with open("attendance.txt", "w") as f:
    for i in range(count):
        name = input(f"Name of student {i + 1}: ")
        f.write(name + "\n")

print("Attendance saved successfully.")
```

Entering 3, then Ada, Bola, Chidi:

Attendance saved successfully.

and `attendance.txt` holds:

Ada  
Bola  
Chidi

`write()` adds **no** newline of its own, unlike `print`. Leave off the + `"\n"` and the whole register lands on one line as `AdaBolaChidi`. The file is opened once, outside the loop, for the reason part (c) just demonstrated.

#### ANSWER · Q5 (E)

scores.py

```
f = open("scores.txt", "w")

while True:
    name = input("Name (or 'done'): ")
    if name == "done":
        break
    score = input("Score: ")
    f.write(name + ", " + score + "\n")

f.close()

f = open("scores.txt", "r")
total = 0
count = 0
passes = 0

for line in f:
    name, raw = line.strip().split(", ")
    score = int(raw)
    total += score
    count += 1
    if score >= 50:
        passes += 1
    print(f"{name}: {score}")

f.close()
print(f"Average: {total / count:.2f}")
print("Passed:", passes)
```

Entering Ada/72, Bola/45, Chidi/58, then done:

Ada: 72  
Bola: 45  
Chidi: 58  
Average: 58.33  
Passed: 2

- **Bola fails.** A pass is 50 **and above**, and 45 is not. Two of the three passed. The average is  $175 / 3 = 58.33$ , which is not a pass mark for anybody: it is just the class average.

- `line.strip()` before `split(", ")`, or the newline sticks to the score and `int("72\n")`... in fact survives, because `int()` ignores surrounding whitespace. Strip anyway: relying on that is a habit that breaks the moment you compare the text instead of converting it.
- **Split on ", " including the space**, because that is exactly how you wrote it. Split on ", " and the score arrives as " 72". The file format and the parser are one decision, made twice.

## Question Six

### MODEL ANSWER

Question Six

Write your full answer to **Question Six** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

ANSWER · Q6 (A)

i. A **table** is one kind of thing being stored, like products. A **row** is one of them: one product. A **column** is one fact held about every one of them: the price. A table is a grid, rows down, columns across.

ii. `conn.commit()` makes your changes **permanent**. Until you call it, inserts live in an open transaction and belong only to your connection. Forget it and the program runs, prints nothing wrong, exits, and **the database is unchanged**. No error is ever raised. It is the single most common way to lose work in this topic.

iii. `fetchone()` returns **one** row and moves on, or `None` if there are none left. `fetchall()` returns **every** remaining row, as a list of tuples, and an empty list if there are none. Use `fetchone()` when you expect a single answer, `fetchall()` when you want to loop over the lot.

ANSWER · Q6 (B)

### Two errors.

1. `VALUE` should be **VALUES**. This is SQL, not Python, so Python cannot help you: the string is perfectly good Python and the database is the one that objects. `OperationalError: near "VALUE": syntax error`
2. `conn.close` has **no brackets**, so the connection is never closed. The third appearance of this fault in two mocks. Note that the program never even reaches it here: the error above stops everything first.

fixed

```
import sqlite3

conn = sqlite3.connect("shop.db")
cursor = conn.cursor()
fields = "(id INTEGER PRIMARY KEY, name TEXT, price INTEGER)"
cursor.execute("CREATE TABLE IF NOT EXISTS products " + fields)
cursor.execute("INSERT INTO products (name, price) VALUES ('Pen', 150)")
```



```
conn.commit()
conn.close()
```

IF NOT EXISTS is added for a reason worth a mark on its own: run the original twice and the second run dies with `OperationalError: table products already exists`. A program that only works once is not finished.

#### ANSWER · Q6 (C)

i. It opens `shop.db`, asks for the name column of every row in `products`, prints all the rows it gets back, and closes the connection.

ii. With the single Pen row from (b) in the table:

```
[('Pen',)]
```

A **list of tuples**, one tuple per row, even though only one column was asked for. That trailing comma in `('Pen',)` is Python's way of writing a tuple of one item, and it is not a typo. To print plain names you would loop and take `row[0]`.

iii. Three assumptions:

1. `shop.db` can be opened. `connect()` **creates** the file if it is absent, so this never fails, and that is the trap.
2. A table called `products` exists in it, with a `name` column. If not: `OperationalError: no such table: products` which is exactly what you get if `connect()` just made you a fresh, empty database because you misspelled the filename.
3. The rows were **committed** by whoever inserted them. Uncommitted inserts from another connection are invisible here.

#### ANSWER · Q6 (D)

`inventory.py`

```
import sqlite3

conn = sqlite3.connect("inventory.db")
cursor = conn.cursor()
fields = "(id INTEGER PRIMARY KEY, name TEXT, price INTEGER)"
cursor.execute("CREATE TABLE IF NOT EXISTS products " + fields)

for i in range(3):
    name = input("Product name: ")
    price = int(input("Price: "))
    cursor.execute("INSERT INTO products (name, price) VALUES (?, ?)", (name, price))

conn.commit()
conn.close()
print("Three products saved.")
```

Three products saved.

- **The ? placeholders.** Pass the values as a tuple second and let sqlite3 put them in. Build the SQL by gluing strings together and a product named O'Brien ends your statement early and crashes it.
- **id is not inserted.** INTEGER PRIMARY KEY fills itself in: 1, 2, 3.
- **conn.commit() is not optional.** Leave it out and this program prints "Three products saved." and saves nothing at all.

#### ANSWER · Q6 (E)

The sentinel loop from Q3(e), with the database from (d) behind it.

inventory\_any.py

```
import sqlite3

conn = sqlite3.connect("inventory.db")
cursor = conn.cursor()
fields = "(id INTEGER PRIMARY KEY, name TEXT, price INTEGER)"
cursor.execute("CREATE TABLE IF NOT EXISTS products " + fields)

while True:
    name = input("Product name (or 'done'): ")
    if name == "done":
        break
    price = int(input("Price: "))
    cursor.execute("INSERT INTO products (name, price) VALUES (?, ?)", (name, price))

conn.commit()

cursor.execute("SELECT * FROM products")
for row in cursor.fetchall():
    print(row)

conn.close()
```

Entering Pen/150, Book/1200, then done:

```
(1, 'Pen', 150)
(2, 'Book', 1200)
```

**Commit before you select.** The order is: loop, commit, then read back. And close() comes last of all, once: put it inside the loop and the second product is inserted through a closed connection.

SELECT \* gives every column, so each row arrives as the full tuple including the id the database made for you. That is why these tuples have three items and Q6(c)'s had one.

## After you have marked it

## Mock Paper Three

## Question One

### MODEL ANSWER

Question One

Write your full answer to **Question One** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

#### ANSWER · Q1 (A) THE OR OPERATOR

**or** is a boolean operator that joins two conditions and is true when **at least one** of them is true. It is false only when both are false, the mirror of and.

```
>>> age = 15
>>> age < 13 or age > 65
False
>>> age < 18 or age > 65
True
```

Like and, it short-circuits: if the left is already true, Python does not bother looking at the right, because the answer is settled.

#### ANSWER · Q1 (B) STRINGS AND METHODS

A **string** is an ordered, immutable sequence of characters in quotes. Four methods, each returning a new string:

| Method   | Does             | Example          | Gives      |
|----------|------------------|------------------|------------|
| upper()  | Capitals         | "go".upper()     | 'GO'       |
| strip()  | Trims ends       | " hi ".strip()   | 'hi'       |
| split()  | Splits to a list | "a b".split()    | ['a', 'b'] |
| count(x) | Counts x         | "aba".count("a") | 2          |

#### ANSWER · Q1 (C) THE BOOK DICTIONARY

The keys are 'title', 'author', 'year' and 'Genre'. The faults are all case mismatches:

- `book['Title']` and `book['Author']`: the keys are lower case. `KeyError`, and 'Title' stops the program first.
- `book['genre']` twice (line 5 and the `del`): the key is 'Genre' with a capital G.
- `book['Year'] = 1959` makes a new key 'Year' instead of updating 'year'. Silent, and wrong.

Corrected so the case matches, it prints:

```
Title: Things Fall Apart
Author: Achebe
Genre: Fiction
Updated Book:
{'title': 'Things Fall Apart', 'author': 'Achebe', 'year': 1959, 'pages': 209}
```

'year' is updated to 1959, 'pages' is added, and 'Genre' is deleted.

#### ANSWER · Q1 (D) SIMPLE INTEREST

```
P = float(input("Principal: "))
R = float(input("Rate (%): "))
T = float(input("Time (years): "))

interest = P * R * T / 100

print(f"Interest: {interest:.2f}")
print(f"Total amount: {P + interest:.2f}")
```

For 50000 at 8% over 3 years:

Interest: 12000.00  
Total amount: 62000.00

#### ANSWER · Q1 (E) TEMPERATURE BAND

```
C = float(input("Celsius: "))
F = C * 9 / 5 + 32

if F <= 32:
    band = "Freezing"
elif F < 60:
    band = "Cold"
elif F < 85:
    band = "Warm"
else:
    band = "Hot"

print(f"Fahrenheit: {F:.1f}")
print("Band:", band)
```

For 25 degrees Celsius:

Fahrenheit: 77.0  
Band: Warm

77 is below 85 but not below 60, so the ladder lands on Warm. The order high-to-low is what makes the bare bounds work.

## Question Two

### MODEL ANSWER

Question Two

Write your full answer to **Question Two** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

#### ANSWER · Q2 (A) CONDITION-CONTROLLED LOOP

A **condition-controlled loop** runs an unknown number of times, repeating as long as a condition stays true and stopping when it turns false. In Python it is the `while` loop. You use it when you do not know in advance how many passes are needed, such as reading until a sentinel.

```
total = 0
while total < 100:
    total += int(input("Add: "))
```

It contrasts with a count-controlled `for` loop, which runs a known, fixed number of times.

#### ANSWER · Q2 (B) CONTROL STRUCTURES

- **Sequence:** statements run in order, top to bottom. `x = 1; y = 2; print(x + y)`
- **Selection:** a choice with `if/elif/else`. `if age >= 18: print("Adult")`
- **Repetition:** a block many times, with `for` or `while`. `while n > 0: n -= 1`

#### ANSWER · Q2 (C) THE EVENS LOOP

Four errors, three of them syntax:

1. **No colon** after the `for` line.
2. `if i % 2 = 0` uses `=` (assignment) where it needs `==` (comparison).
3. `total + i` computes and throws the result away; it should be `total += i`.
4. `print("Sum of evens:" total)` is missing the comma between its two arguments. And `range(1, 10)` stops at 9, so 10 is missed: use `range(1, 11)`.

fixed

```
total = 0
for i in range(1, 11):
    if i % 2 == 0:
        total += i
print("Sum of evens:", total)
```

Sum of evens: 30

The evens 2, 4, 6, 8, 10 add to 30. Answer 20 and you kept the off-by-one in the range.

#### ANSWER · Q2 (D) THE PIN PROGRAM

system.py

```
while True:
    pin = input("PIN: ")
    if pin == "2468":
        print("PIN Accepted")
        break
    print("Wrong PIN")

age = int(input("Age: "))
if age < 0 or age > 120:
    print("Error: age must be 0 to 120")
```

```

else:
    if age < 13:
        print("Child")
    elif age < 20:
        print("Teenager")
    elif age < 65:
        print("Adult")
    else:
        print("Senior")

start = int(input("Count down from: "))
for i in range(start, 0, -1):
    print(i)
print("Liftoff!")

total = 0
count = 0
while True:
    n = int(input("Number (-1 to stop): "))
    if n == -1:
        break
    total += n
    count += 1
print(f"Average: {total / count:.2f}")

```

PIN 1111, then 9999, then 2468 gives "Wrong PIN" twice then "PIN Accepted". Age 30 is an Adult; age 200 gives the error. A countdown from 5 prints 5 4 3 2 1 then "Liftoff!". Numbers 20, 40, 60, then -1 give:

Average: 40.00

`range(start, 0, -1)` counts down and stops before 0, so it ends at 1. The step of -1 is what reverses it.

## Question Three

### MODEL ANSWER

### Question Three

Write your full answer to **Question Three** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

#### ANSWER · Q3 (A) FIVE BENEFITS

Reuse; readability; maintenance; decomposition into small testable pieces; and abstraction, using a function by what it does rather than how. A sixth if needed: collaboration, since a team can split the functions between them.

#### ANSWER · Q3 (B) FUNCTIONS, PARAMETER VS ARGUMENT

A **function** is a named block of code that runs when called. Functions matter for reuse, readability, maintenance and decomposition.

A **parameter** is the name in the definition; an **argument** is the actual value you pass when you call. In `def greet(name)`, `name` is the parameter; in `greet("Ada")`, `"Ada"` is the argument. Parameter is the empty slot, argument is what you drop into it.

#### ANSWER · Q3 (C) THE LIST TRACE

- `[0] * 3` repeats the list, so `a` is `[0, 0, 0]`.
- `b.append([4, 5])` adds the list `[4, 5]` as a **single** item, so `b` is `[1, 2, 3, [4, 5]]`.
- `len(b)` is 4: three numbers and one inner list, which counts as one.

```
a = [0, 0, 0]
b = [1, 2, 3, [4, 5]]
len(b) = 4
```

The trap is expecting `append` to spread `[4, 5]` into two items. It does not: it appends the list whole. That is why `len(b)` is 4, not 5. To spread, you would use `b += [4, 5]` or `b.extend([4, 5])`.

#### ANSWER · Q3 (D) THE COLLECTIONS PROGRAM

A set for the enrolled (unique, unordered), a tuple for the fixed club details, a set for attendance, a dictionary for points. The two "absent" and "not enrolled" questions are set difference both ways.

club.py

```
enrolled = {"Ada", "Ben", "Chidi", "Dami", "Efe", "Femi"}
details = ("Coding Club", "Lab 3", "Friday", 25)
present = {"Ada", "Ben", "Grace", "Femi", "Chidi"}
points = {"Ada": 88, "Ben": 54, "Chidi": 72,
          "Dami": 45, "Efe": 63, "Femi": 91}

print("Club info:", details)
print("Enrolled:", sorted(enrolled))
print("Enrolled, absent:", sorted(enrolled - present))
print("Present, not enrolled:", sorted(present - enrolled))
print("Points 60+:", sorted(nm for nm, p in points.items() if p >= 60))
```

```
Club info: ('Coding Club', 'Lab 3', 'Friday', 25)
Enrolled: ['Ada', 'Ben', 'Chidi', 'Dami', 'Efe', 'Femi']
Enrolled, absent: ['Dami', 'Efe']
Present, not enrolled: ['Grace']
Points 60+: ['Ada', 'Chidi', 'Efe', 'Femi']
```

Dami and Efe are enrolled but were absent; Grace turned up without enrolling. sorted around each set makes the printed order stable and markable.

## Question Four

### MODEL ANSWER

### Question Four

Write your full answer to **Question Four** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

#### ANSWER · Q4 (A) LISTS ARE MUTABLE

**Mutable** means a list can be changed in place after it is made: you can reassign an item, add one, or remove one, and it is still the same list.

```
>>> nums = [10, 20, 30]
>>> nums[0] = 99
>>> nums
[99, 20, 30]
```

**The reason it matters:** a string is immutable and refuses `s[0] = "x"` with a `TypeError`, while a list allows it. So a list is the structure you reach for when the data must change as the program runs.

#### ANSWER · Q4 (B) THE HALF FUNCTION

(i) `result = half(10)`. (ii) `half(10)` returns `10 / 2`, and `/` always gives a float, so:

```
result = 5.0
```

Note `5.0`, not `5`: true division `/` returns a float even when the result is whole. That decimal point is worth a mark.

#### ANSWER · Q4 (C) THE LIBRARY DATABASE

library.py

```
import sqlite3

conn = sqlite3.connect("library.db")
cursor = conn.cursor()
cursor.execute("""CREATE TABLE IF NOT EXISTS book
                  (BookID INTEGER PRIMARY KEY, Title TEXT,
                   Author TEXT, Copies INTEGER)""")

books = [(1, "Half of a Yellow Sun", "Adichie", 4),
         (2, "Things Fall Apart", "Achebe", 7),
         (3, "The Joys of Motherhood", "Emecheta", 3),
         (4, "Arrow of God", "Achebe", 5),
         (5, "Efuru", "Nwapa", 2)]

cursor.executemany("INSERT INTO book VALUES (?, ?, ?, ?)", books)
conn.commit()

print("All books:")
cursor.execute("SELECT * FROM book")
for row in cursor.fetchall():
    print(row)

print("By Achebe:")
cursor.execute("SELECT * FROM book WHERE Author = ?", ("Achebe",))
for row in cursor.fetchall():
    print(row)
```



```

cursor.execute("SELECT COUNT(*), MAX(Copies), AVG(Copies) FROM book")
titles, most, average = cursor.fetchone()
print("Titles:", titles, " Most:", most, " Average:", average)

cursor.execute("UPDATE book SET Copies = ? WHERE Title = ?", (6, "Efuru"))
conn.commit()
cursor.execute("DELETE FROM book WHERE BookID = ?", (3,))
conn.commit()
conn.close()

```

All books:

```

(1, 'Half of a Yellow Sun', 'Adichie', 4)
(2, 'Things Fall Apart', 'Achebe', 7)
(3, 'The Joys of Motherhood', 'Emecheta', 3)
(4, 'Arrow of God', 'Achebe', 5)
(5, 'Efuru', 'Nwapa', 2)

```

By Achebe:

```

(2, 'Things Fall Apart', 'Achebe', 7)
(4, 'Arrow of God', 'Achebe', 5)

```

Titles: 5 Most: 7 Average: 4.2

Copies is INTEGER, so the rows show 4 and 7, not 4.0. The average, 4.2, is a real number because an average can be fractional even when the column is whole. After the update Efuru holds 6, and after the delete BookID 3 is gone.

## Question Five

### MODEL ANSWER

### Question Five

Write your full answer to **Question Five** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

**ANSWER · Q5 (A), (B), (C)**

**(a)** File handling is reading from and writing to files on disk, so data lasts after the program ends, using `open()`, the read and write methods, and `close()`.

**(b)** Four modes:

| Mode | Means          | Exists         | Missing           |
|------|----------------|----------------|-------------------|
| "r"  | Read           | Reads          | FileNotFoundError |
| "w"  | Write          | Erased first   | Created           |
| "a"  | Append         | Adds at end    | Created           |
| "r+" | Read and write | Both, no erase | FileNotFoundError |

**(c)** Reading takes data out and changes nothing. Writing ("w") puts data in and **erases** whatever was there first. One looks, the other replaces.

**ANSWER · Q5 (D) THE INVENTORY FILE**

```

filename = "inventory.txt"

with open(filename, "w") as f:
    for i in range(3):
        name = input("Item: ")
        code = input("Code: ")
        qty = input("Quantity: ")
        f.write(f"Item: {name}, Code: {code}, Quantity: {qty}\n")

with open(filename, "a") as f:
    for i in range(2):
        name = input("Item: ")
        code = input("Code: ")
        qty = input("Quantity: ")
        f.write(f"Item: {name}, Code: {code}, Quantity: {qty}\n")

print("All items:")
with open(filename, "r") as f:
    for line in f:
        print(line.strip())

print("First two:")
with open(filename, "r") as f:
    for i, line in enumerate(f):
        if i >= 2:
            break
        print(line.strip())

```

Entering Pencil/P01/120, Eraser/E01/75, Ruler/R01/60, then two more, the reads give:

```

All items:
Item: Pencil, Code: P01, Quantity: 120
Item: Eraser, Code: E01, Quantity: 75
Item: Ruler, Code: R01, Quantity: 60
First two:
Item: Pencil, Code: P01, Quantity: 120
Item: Eraser, Code: E01, Quantity: 75

```

The append opens with "a", so the first three survive. Had it opened with "w", the file would have been wiped before the extra items were added.

## Question Six

### MODEL ANSWER

### Question Six

Write your full answer to **Question Six** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

#### ANSWER · Q6 (A) NESTED INDEXING

- `data[1][1][2]`: `data[1]` is `[300, [400, 500, 600]]`, its `[1]` is `[400, 500, 600]`, and `[2]` of that is `600`.
- `len(data[1])`: `data[1]` holds two items, so `2`.
- `data[2][0][1]`: `data[2]` is `[[700, 800], 900]`, its `[0]` is `[700, 800]`, and `[1]` of that is `800`.

`600`

`2`

`800`

Follow the brackets strictly left to right, writing down each layer as you peel it.

#### ANSWER · Q6 (B) THE VOID SUMMARY

`results.py`

```
def print_result_summary(student, total_score, pass_mark):
    print(f"=== Result for {student} ===")
    print("Total Score:", total_score)
    if total_score >= pass_mark:
        print("Result: PASS")
    else:
        print("Result: FAIL")

name = input("Student name: ")
subjects = int(input("Number of subjects: "))
total_score = 0
for i in range(subjects):
    total_score += int(input(f"Score for subject {i + 1}: "))

print_result_summary(name, total_score, 250)
```

For Bola with five scores 70, 65, 80, 55, 70 (total 340):

```
=== Result for Bola ===
Total Score: 340
Result: PASS
```

340 clears the 250 pass mark. The function only prints and returns nothing, which is what "void" means: called for its effect, not for a value.

#### ANSWER · Q6 (C) THE CYLINDER TANK

`tank.py`

```
import math

def volume(r, h):
    return math.pi * r ** 2 * h

radius = float(input("Radius: "))
height = float(input("Height: "))
```

```
v = volume(radius, height)
print(f"Volume: {v:.2f}")
if v > 1000:
    print("Large tank")
else:
    print("Small tank")
```

For radius 7 and height 10:

Volume: 1539.38

Large tank

$\text{math.pi} * 7 ** 2 * 10$  is  $\text{math.pi} * 49 * 10 = 1539.38$ . The  $** 2$  binds before the multiplications, so the radius is squared first, no brackets needed. 1539.38 exceeds 1000, so it is a large tank.